

MCP 이해와 활용

2026.09.29(화)

KISTI 이 준

차 례

❖ 오전

- MCP의 이론 및 관련 기술에 대한 소개
- 실습 환경 구축과 에러시 대응 방안, 간단한 실습

❖ 오후

- 외부에서 MCP 서버 가져다 쓰는 방식 실습
- MCP 코드 작성 및 직접 설정 방식 위주의 예제 실습
- 반복업무 자동화 및 활용사례

MCP 이해와 활용

오전 수업 (10:00~11:30)

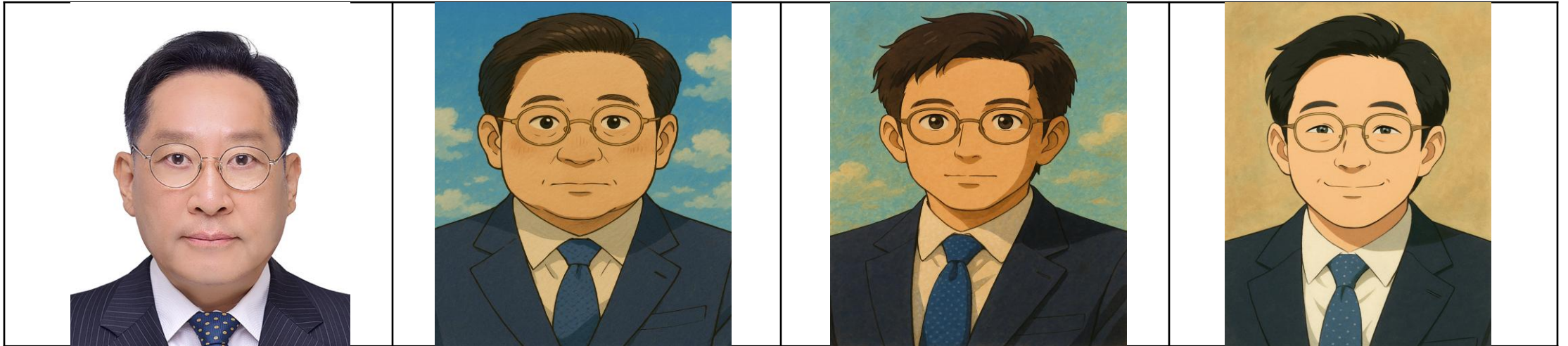
학습목표

- LLM에 대한 전반적인 리뷰 및 MCP 등장 배경
- MCP의 기반 기술에 대한 이해 (클라이언트/서버 환경, 아키텍처, 구성 요소 등)
- MCP 실습 환경 구축 관련, 각 기술이 왜 필요하며 어떤 역할을 담당하는가
- 작성된 MCP 서버 가져다 쓰는 방식 실습 vs MCP 코드 작성 및 직접 설정 방식
- (최종) LLM 활용시 MCP가 무엇이고 실무에 어떻게 사용하면 되겠다는 전반적인 이해를 통해 향후 학습 방향에 대한 실마리 제공

당부의 말씀

- MCP는 여러 기술을 활용하는 프로토콜입니다. 단 본 과정은 MCP와 관련 기술에만 집중합니다 (No!! 피노키오)
- 실습 환경 구축이 안되거나 실습이 잘 안되면 도움을 요청하세요.
- 구글 아이디 준비하세요. 실습을 위해 필요합니다.
- 클로드 무료 버전을 쓰면 쓸 수 있는 토큰이 제한적이라 실습 도중 사용이 중지될 수 있으니 주어진 실습에만 집중해 주세요.^^
- 더 나은 강의가 되도록 피드백을 주세요. 설문 문항에 의견 주시면 반영하여 개선 하겠습니다.
- AI 기술은 매우 빠르게 변화하고 있습니다. 세부적인 기능의 변화보다는 큰 흐름을 이해하고 어떻게 내 업무에 활용하면 유용할 지에 대해 생각해 보시는 시간이 되시길 바랍니다. 감사합니다.

0-1 강사소개



- 강사 이름: 이 준 (rjlee98@kisti.re.kr/042-869-0675)
- 전공: 컴퓨터공학박사(분산컴퓨팅 전공, 영국 맨체스터대학교)
- 경력: 한국과학기술정보연구원 책임연구원 (현재)
국립한밭대학교 정보통신공학과 겸임교수
- 관심 분야: MCP, AI Agent, Vibe Coding, 하네스 엔지니어링 등

0-2 참고문헌

- LLM을 활용한 실전 AI 애플리케이션 개발, 책만, 허정준, 2024.7
- 10분만에 따라하는 Claude MCP 업무 자동화 혁신 가이드, 리코멘드, 이호준, 2025.7
- 이게 되네? 클로드 MCP 미친 활용법 27제, 골든래빗, 박현규, 2025.7
- 나만의 MCP 서버 만들기 with 커서 AI, 길벗, 서지영, 2025.7
- 클로드 코워크 with 스킵, 플러그인, 한빛미디어, 클리커, 강민혁, 2026.3

0-3 강의 진행

❖ 오전

- MCP의 이론 및 관련 기술에 대한 소개
- 실습 환경 구축과 에러시 대응 방안, 간단한 실습

❖ 오후

- 외부에서 MCP 서버 가져다 쓰는 방식 실습
- MCP 코드 작성 및 직접 설정 방식 위주의 예제 실습
- 반복업무 자동화 및 활용사례

1-1. MCP 개념과 동작 방식

1) MCP의 개념

○ LLM 모델의 발전과 한계

- LLM 모델의 비약적 발전 (2024-2026년)

- 2024년 초: OpenAI의 GPT-4 Turbo의 128K 컨텍스트 윈도우 지원 등 복잡한 문서 분석까지 가능
- 2024년 중반: Anthropic의 Claude 3.5 Sonnet은 코딩, 분석, 창작 전 영역에서 벤치마크를 갱신
- 2024년 말: Google의 Gemini 2.0은 멀티모달 능력을 획기적으로 향상
- 2025년: Claude 4, GPT-5 등 차세대 모델들이 등장하며, 추론(reasoning) 능력이 전문가 수준에 근접
- 2026년: LLM은 대부분의 지식 작업에서 인간과 대등하거나 그 이상의 성능을 갱신
- 특히 2026년 4월 차세대 AI 모델로 발표된 Anthropic의 **클로드 미토스(Claude Mythos)**는 전문가 수준의 소프트웨어 보안 취약점 탐지 능력과 취약점 발견 후 공격 코드까지 스스로 작성 가능하다는 점, 취약점 여러 개를 조합해 더 큰 공격 설계가 가능하다는 특징들로 인하여 공개가 보류 되는 등 뛰어난 사이버 공격 능력으로 인하여 '**보안 핵폭탄**'으로 불림

1-1. MCP 개념과 동작 방식

- LLM의 한계 (문제상황 시나리오)

- 시나리오 1: 매출 데이터 분석

사용자: "이번 달 매출 데이터를 분석해서 보고서를 만들어줘."

AI의 속마음: "매출 데이터가 어디 있는지 모르겠고,

데이터베이스에 접근할 수도 없고,

파일을 읽을 수도 없네...

일단 가상의 예시로 보고서 형식만 만들어줄게."

AI의 실제 답변: "매출 데이터를 직접 확인할 수는 없지만,

일반적인 매출 보고서 양식을 알려드리겠습니다..."

1-1. MCP 개념과 동작 방식

- LLM의 한계 (문제상황 시나리오)

- 시나리오 2: 이메일 발송

사용자: "김 부장님께 회의 일정 변경 메일 보내줘."

AI의 속마음: "이메일 시스템에 접근할 수 없어..."

김 부장님 이메일 주소도 모르고...

보내는 것은 불가능해."

AI의 실제 답변: "메일을 직접 보낼 수는 없지만,

이메일 초안을 작성해드리겠습니다..."

1-1. MCP 개념과 동작 방식

- LLM의 한계 (문제상황 시나리오)

- 시나리오 3: 일정 관리

사용자: "다음 주 수요일 오후 3시에 팀 미팅 잡아줘."

AI의 속마음: "캘린더에 접근할 수 없어..."

팀원들의 일정도 확인할 수 없고..."

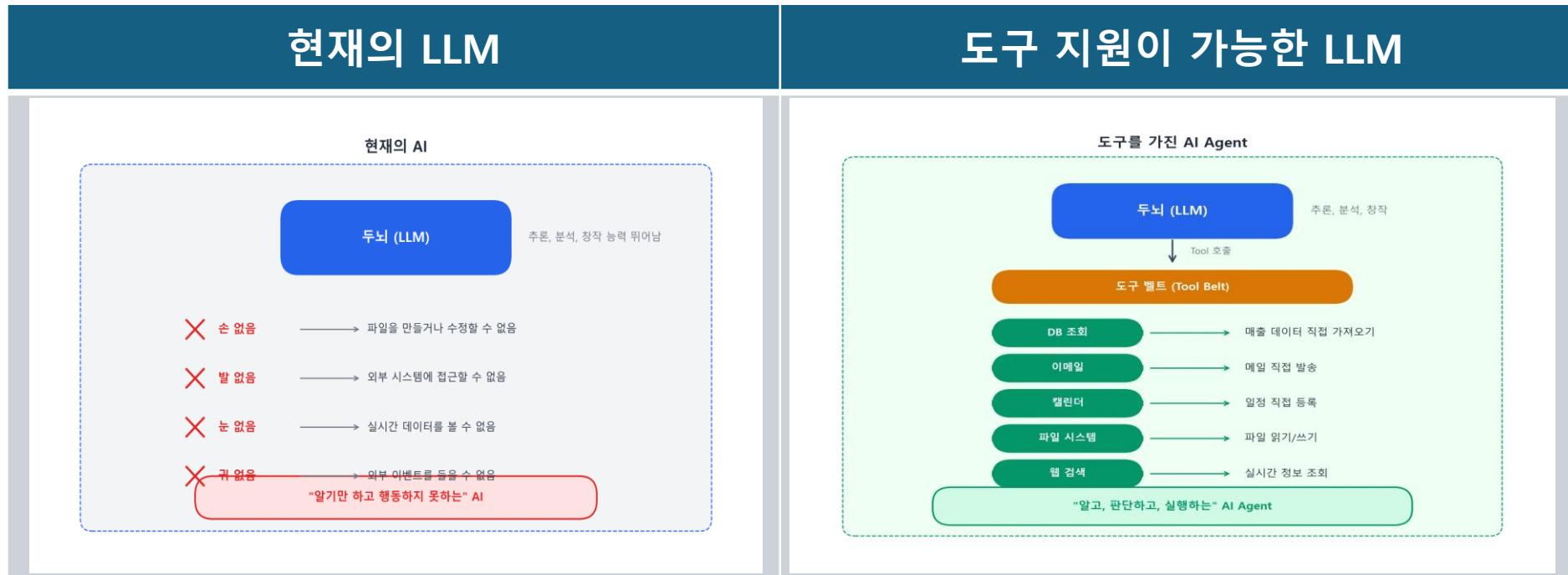
AI의 실제 답변: "캘린더에 직접 일정을 추가할 수는 없지만,
일정 초대 메일 초안을 작성해드릴 수 있습니다..."

- 공통 패턴: LLM의 답변은 항상 "~할 수는 없지만, ~ 해드리겠습니다." 로 한계 노출

1-1. MCP 개념과 동작 방식

○ LLM의 한계를 극복하기 위한 방안

- LLM의 한계(시나리오 참조)를 극복하기 위한 대안으로는 외부 세계와 상호 작용할 수 있는 “**도구(Tool)**”가 필요
- 즉 데이터베이스를 조회하고, 이메일을 전송하고, 파일을 수정하고, API를 호출하는 등 구체적인 행동을 수행할 수 있는 능력이 필요
- LLM이 외부함수를 호출할 수 있게 하는 기능을 “Tool Use(Anthropic)” 또는 “Function Calling(오픈AI)”이라고 하며, 개념은 동일



1-1. MCP 개념과 동작 방식

○ Funtion Calling과 MCP의 비교

구분	Funtion Calling	MCP
맥락관리	stateless(기억없음)	Stateful(세션간 기억 유지)
도구관리	미리 정의 필요	동적 탐색 가능
작업구조	단선적 실행	다중 에이전트 협업 가능
시스템 구조	하나의 앱에 결합	클라이언트-서버 모듈화
적합한 사례	단순 자동화, 프로토타입	기업 시스템, 자율적 에이전트

출처: Function Calling vs. MCP: 차세대 AI 팀이 꼭 알아야할 핵심 차이점(<https://digitalbourgeois.tistory.com/1887>)

1-1. MCP 개념과 동작 방식

○ API 래핑(API Wrapping)의 한계: $N \times M$ 문제

- MCP 이전 LLM에 도구를 연결하는 방식은 API 래핑이 대부분이었음
- API 래핑이란 외부 시스템의 API를 LLM 모델 플랫폼이 이해할 수 있는 형태로 감싸서 제공하는 방식으로 예를 들면,
 - ChatGPT에서 Slack을 사용하려면 → ChatGPT용 Slack 플러그인 개발
 - Claude에서 Slack을 사용하려면 → Claude용 Slack 통합 코드 개발
 - Gemini에서 Slack을 사용하려면 → Gemini용 Slack 연동 모듈 개발
- 같은 시스템을 연결하는데도 LLM 플랫폼별로 별도의 통합 코드를 개발해야 함

1-1. MCP 개념과 동작 방식

○ N × M 문제 예시

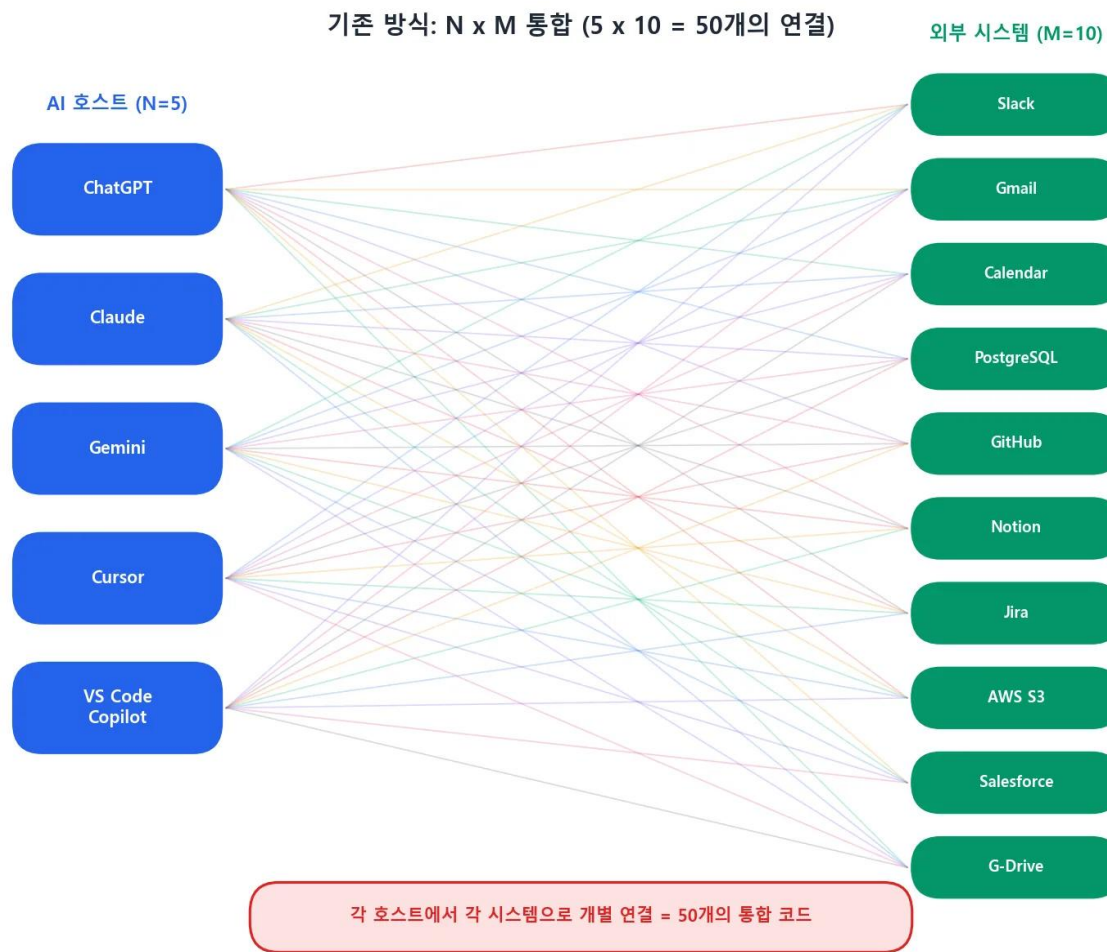
- 현재 주요 LLM 모델을 예로 들면,

1. ChatGPT
2. Claude Desktop
3. Gemini
4. Cursor IDE
5. VS Code Copilot (N = 5)

- 연결해야 할 외부 시스템은

1. Slack (메신저)
2. Gmail (이메일)
3. Google Calendar (일정)
4. PostgreSQL (데이터베이스)
5. GitHub (코드 저장소)
6. Notion (문서)
7. Jira (프로젝트 관리)
8. AWS S3 (파일 저장소)
9. Salesforce (CRM)
10. Google Drive (파일) (M = 10)

- 기존 방식으로는 $N \times M = 5 \times 10 = 50$ 개의 통합 코드가 필요



1-1. MCP 개념과 동작 방식

- N × M 방식, 무엇이 문제인가?

(1) 유지보수

상황	결과
Slack API가 v2에서 v3으로 업데이트	N개의 호스트에서 각각 Slack 연동 코드 수정
새로운 AI 호스트 등장 (예: 새 IDE)	기존 M개 시스템 전체와 새로 통합
새로운 외부 시스템 추가 (예: Linear)	기존 N개 호스트 전체에 새로 통합
보안 취약점 발견	N×M개 코드 전부 점검 및 패치
인증 방식 변경	관련 통합 코드 전체 수정

(2) LLM 모델별로 도구 연결 방식이 완전히 다름

AI 플랫폼	도구 연결 방식	도구 정의 형식
OpenAI	Function Calling (독자 규격)	OpenAI 전용 JSON 스키마
Anthropic	Tool Use (독자 규격)	Anthropic 전용 JSON 스키마
Google	Gemini Function Calling	Google 전용 형식
LangChain	Tools/Agents 프레임워크	LangChain 전용 래퍼
LlamaIndex	Tool Abstractions	LlamaIndex 전용 인터페이스

1-1. MCP 개념과 동작 방식

○ 해결 방안: $N \times M$ 에서 $N+M$ 으로

- MCP의 핵심가치는 통합비용을 극적으로 감소시키는 것임

- 기존(MCP 없음): N 개 호스트 $\times$ M 개 시스템 = $N \times M$ 개 통합 코드
- MCP 도입 후: N 개 호스트의 MCP 클라이언트 + M 개의 MCP 서버 = $N+M$ 개 코드

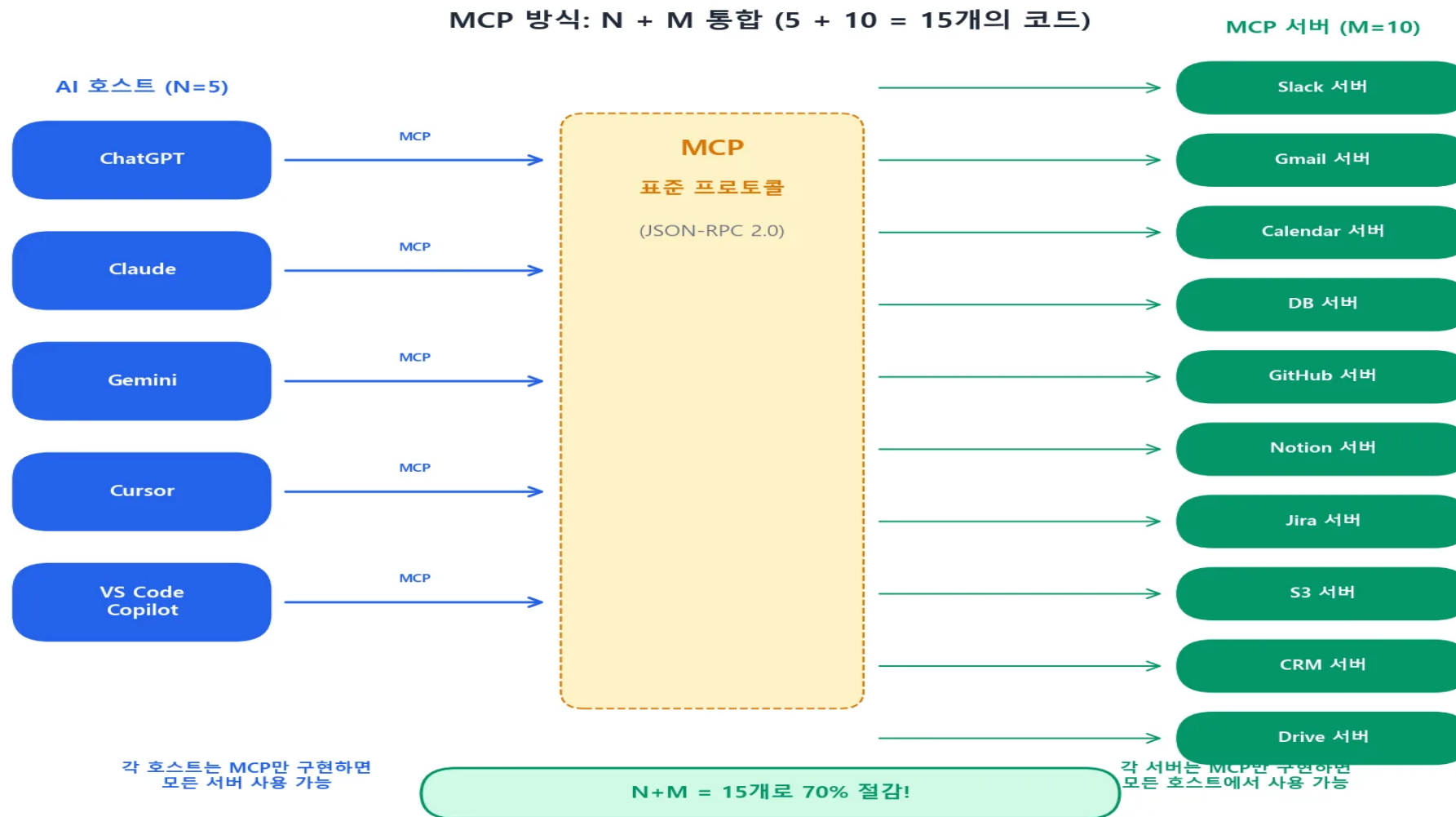
- 앞의 예를 다시 적용하면,

- 기존: $5 \times 10 = 50$ 개
- MCP: $5 + 10 = 15$ 개 (70% 감소)

- 규모 증가에 따른 감소 효과

N (AI 호스트)	M (외부 시스템)	기존 ($N \times M$)	MCP ($N+M$)	절감률
5	10	50	15	70%
10	50	500	60	88%
20	100	2,000	120	94%
50	200	10,000	250	97.5%

1-1. MCP 개념과 동작 방식



1-1. MCP 개념과 동작 방식

○ LSP에서 MCP로

- 기존 LLM 모델들이 외부 도구를 사용하는 방법이 표준화되어 있지 않아 이를 통합할 목적으로 고안
- AI 모델과 외부 리소스(도구, 데이터)를 유연하게 통합하기 위한 오픈 프로토콜로 제시
- 오픈 프로토콜로써, MCP는 깃허브에 저장소를 두고 프로토콜 스펙을 공개
- 엔트로픽의 데이비드 소리아 파라(David Soria Parra)와 저스틴 스파 섬머스(Justin Spahr-Summers)가 클로드 데스크톱을 개발자 도구와 더 잘 연동시켜려고 고민하던 중 LSP Project에서 영감을 받아 개발
- LSP(Language Server Protocol)은 MS의 VS Code IDE 개발 과정에서 탄생
- IDE에서 프로그램 언어를 지원해야 하는 기능(자동완성, 함수와 클래스 정의로 이동하기, 검색하기, 문법 오류 검사, 코드 포매팅 등)을 프로그램 언어별로 구현해야 하는 어려움을 해결하기 위한 방안으로 MS는 LSP 프로토콜 개발을 통해 다양한 프로그램 언어 서버와 개발 도구간의 통신 프로토콜을 표준화하는 방식으로 해결
- 같은 맥락에서 AI 모델과 외부 리소스의 관계도 프로토콜 개발로 해결 가능하지 않을까 하는 의도에서 MCP 개발
 - LSP: IDE가 프로그래밍 언어 도구와 상호작용하는 방법을 표준화
 - MCP: AI 애플리케이션이 외부 시스템과 상호작용하는 방법을 표준화

1-1. MCP 개념과 동작 방식

○ LSP vs. MCP 비교

구분	Language Server Protocol (LSP)	Model Context Protocol (MCP)
문제 정의	N 개의 코드 에디터가 M 개의 프로그래밍 언어를 지원해야 하는 문제	N 개의 AI 애플리케이션이 M 개의 외부 도구/데이터를 지원해야 하는 문제
해결책	언어별로 하나의 '언어 서버'를 만들고, 에디터는 LSP 표준에 맞춰 통신	도구/데이터별로 하나의 'MCP 서버'를 만들고, AI 앱은 MCP 표준에 맞춰 통신
결과	모든 에디터가 쉽게 다수의 언어를 지원 가능. 언어 개발자는 한 번만 서버를 만들면 됨.	모든 AI 앱이 쉽게 다수의 도구를 활용 가능. 도구 개발자는 한 번만 서버를 만들면 됨.

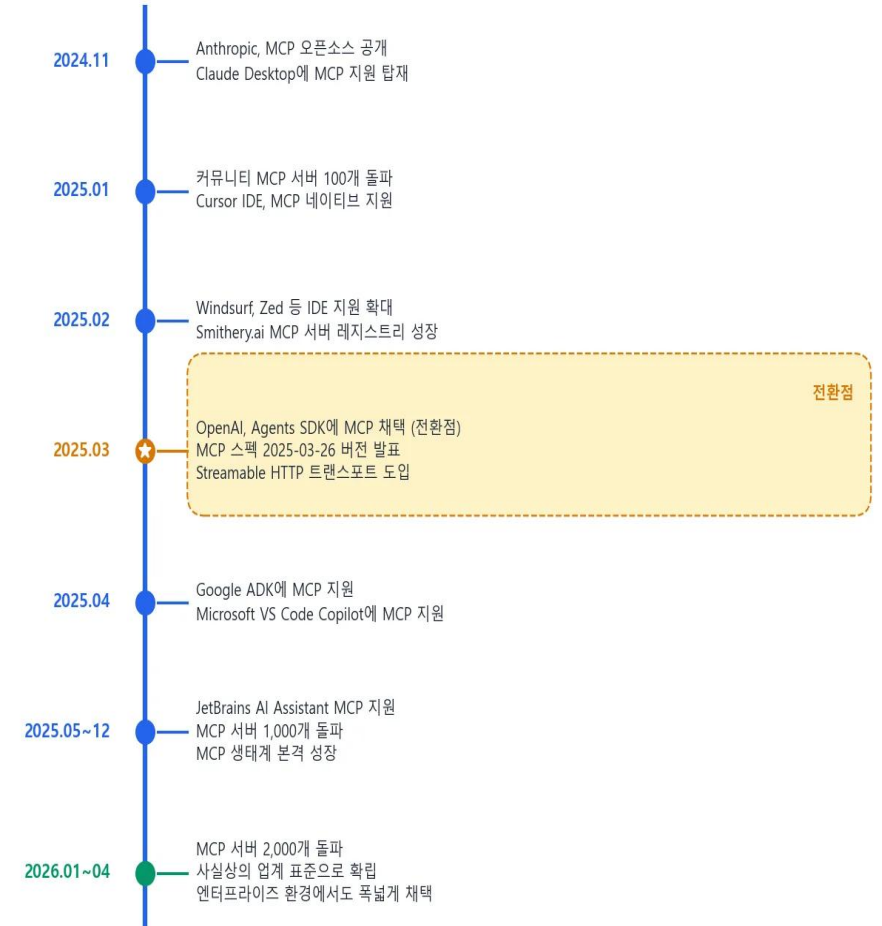
출처: 모델 컨텍스트 프로토콜(MCP): AI 에이전트 시대의 통합 가이드, 채찍피티, 2026.1.19 (<https://wikidocs.net/book/17996>)

1-1. MCP 개념과 동작 방식

○ MCP(Model Context Protocol) 정리

- 2024년 말 Anthropic에서 제안, 클로드 데스크탑, Cursor AI, 스미더리 AI 등에서 채택
- LLM이 외부의 다양한 도구(tool)와 구조화된 방식으로 상호작용하도록 설계된 프로토콜
- [The USB-C for AI Agents - Standardizing Context exchange](#) (링크 참조)
- MCP를 통해 다양한 외부 도구(파일탐색기, API 서버, 클라우드 서비스, 워드 프로세서 등)와의 연동이 가능하고 작업을 자동화하거나 결과를 가져오는 것이 가능
- 클로드 MCP 공식문서 “MCP는 AI 애플리케이션을 위한 USB-C 포트와 같습니다. USB-C 가 다양한 주변기기와 액세서리에 기기를 연결하는 표준화된 방법을 제공하는 것처럼 MCP는 AI 모델을 다양한 데이터 소스와 도구에 연결하는 표준화된 방법을 제공합니다.”

MCP 채택 타임라인



1-1. MCP 개념과 동작 방식

❖ MCP의 주요 특징

1. **표준 프로토콜 정의**: AI 호스트와 도구 서버가 소통하는 방법을 하나로 통일
2. **역할 분리**: "AI 호스트"와 "도구 서버"를 명확히 분리하여, 각자 역할에만 집중
3. **JSON-RPC 2.0 기반**: 검증된 표준 프로토콜 위에 구축하여 안정성과 호환성 확보

2) MCP의 동작원리

○ MCP 아키텍처

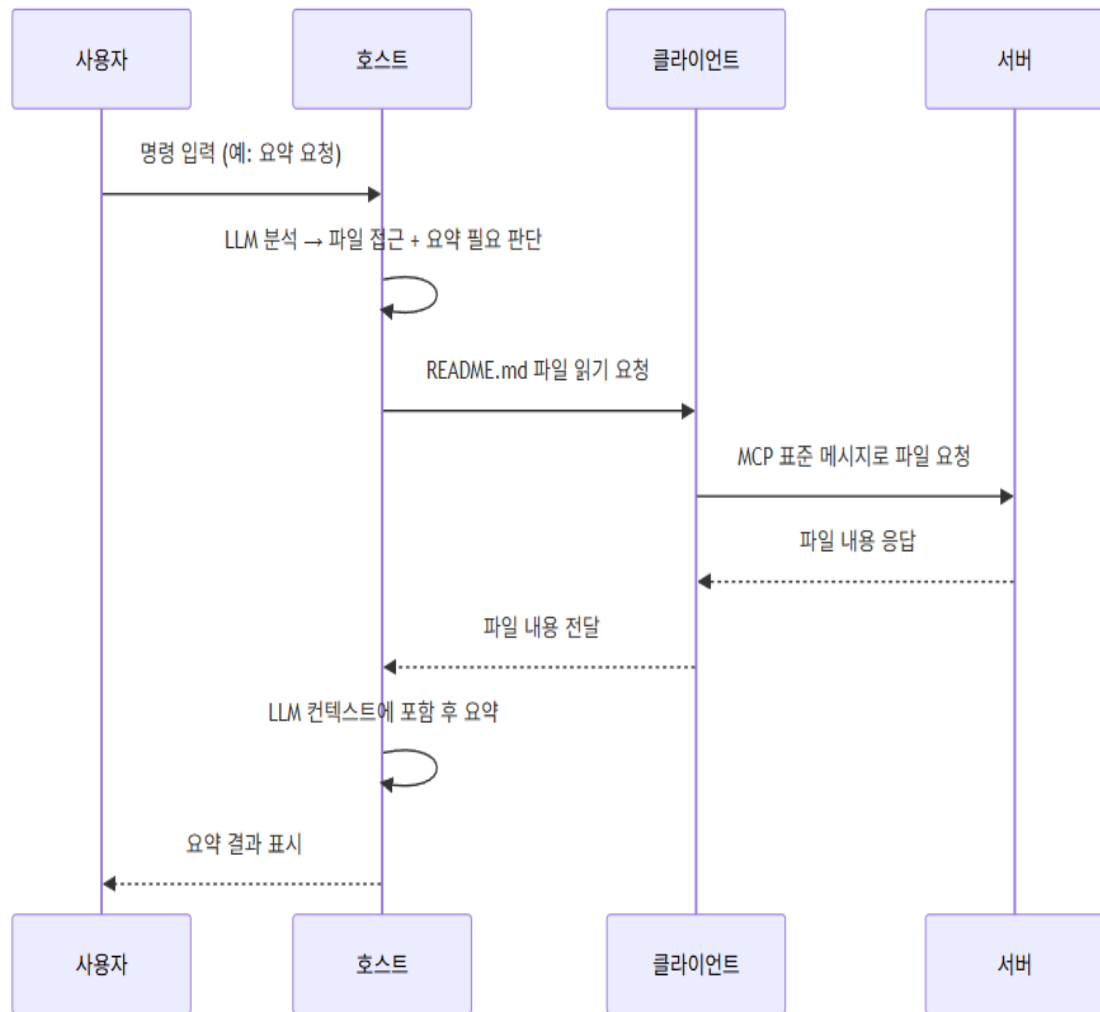
- 기본적으로 서버-클라이언트 아키텍처 방식, 역할을 중심으로 서버는 클라이언트로부터 요청(request)을 받아 처리하고 결과를 클라이언트에게 응답(response)의 형태로 전달
- 서버, 클라이언트는 고정된 개념이 아니라 상대적 개념, 서버가 다른 서버로 요청을 할 경우는 클라이언트 역할을 수행
- 서버-클라이언트 아키텍처의 예: 웹검색시 사용자가 웹브라우저를 통해 서버에 접속하여 요청을 보내면 검색엔진 서버는 요청에 해당하는 검색 결과 페이지를 응답으로 반환
- MCP 시스템은 호스트(Host), 클라이언트(Client), 서버(Server)라는 세가지 핵심 주체로 구성

2) MCP의 동작원리

구분	정의	역할
Host	사용자가 직접 상호작용하는 최상위 AI 애플리케이션, Claud Desktop, Cursor IDR 등이 해당 (오케스트라의 지휘자)	- 중앙 제어: LLM과 통합, 사용자 인터페이스 제공 등 전체 워크플로우를 총괄 - 보안 및 권한 관리: 여러 MCP 서버에 대한 접근권한을 관리하고, 외부 도구 실행 전 사용자 동의를 구하는 등 보안 정책을 시행하는 최종 책임자 - 클라이언트 관리: 내부에 여러 MCP 클라이언트 인스턴스를 생성하고 관리
Client	호스트내에 존재하며, 특정 서버와 1:1로 직접 통신하는 중재자 (충실한 통역자)	- 통신 담당: 호스트의 지시에 따라 서버와 지속적인 연결 세션을 유지하고 JSON-RPC 2.0 형식의 표준화된 메시지를 교환 - 메시지 변환: 호스트의 내부 요청을 서버가 이해할 수 있는 MCP 메시지로 변환하고, 서버의 응답을 다시 호스트가 사용할 수 있는 형태로 변환
Server	특정 데이터 소스(파일시스템, DB)나 외부 도구(GitHub API, Slack API)를 MCP 표준에 맞춰 감싼 (wrapping) 경량 어댑터 (각 분야의 전문가)	- 기능 제공: 자신이 감싸고 있는 자원의 기능을 '프롬프트', '리소스', '도구'라는 표준화된 형태로 외부에 노출 - 실제 작업 수행: 클라이언트로부터 요청을 받으면, 실제 파일 읽기, API 호출 등의 작업을 수행하고 그 결과를 반환

2) MCP의 동작원리

- 데이터 흐름 관점에서의 상호작용



사용자가 "내 프로젝트의 main 브랜치에 있는 README.md 파일 내용을 요약해줘"라고 요청하는 시나리오를 통해 전체 데이터 흐름을 살펴보면,

1. **[사용자 → 호스트]** 사용자가 Claude Desktop(호스트)에 명령을 입력합니다.
2. **[호스트 내부]** 호스트의 LLM이 이 요청을 분석하고, '파일 시스템 접근'과 '텍스트 요약'이 필요하다고 판단합니다.
3. **[호스트 → 클라이언트]** 호스트는 파일 시스템 접근을 담당하는 '파일시스템 MCP 클라이언트'에게 README.md 파일을 읽으라는 지시를 내립니다.
4. **[클라이언트 ↔ 서버]** 클라이언트는 '파일시스템 MCP 서버'에 MCP 표준에 맞는 파일 읽기 요청 메시지를 전송합니다. 서버는 실제 파일을 읽어 그 내용을 응답 메시지에 담아 클라이언트에게 반환합니다.
5. **[클라이언트 → 호스트]** 클라이언트는 서버로부터 받은 파일 내용을 호스트에 전달합니다.
6. **[호스트 → 사용자]** 호스트는 전달받은 파일 내용을 LLM의 컨텍스트에 포함시켜 요약한 뒤, 최종 결과를 사용자에게 보여줍니다.

이처럼 각자의 역할이 명확히 분리되어 있기 때문에, 새로운 도구를 추가하고 싶다면 해당 도구를 위한 '서버'만 개발하면 됩니다. 호스트나 다른 서버들은 전혀 수정할 필요가 없습니다. 이것이 바로 MCP가 'N×M 문제'를 근본적으로 해결하는 방식입니다.

2) MCP의 동작원리

❖ 호스트가 서버와 직접 통신하지 않고 클라이언트를 별도로 둔 이유:

1. **격리(Isolation)**: 각 서버와의 통신이 독립적이므로, 한 서버에 문제가 생겨도 다른 서버에 영향을 주지 않도록 격리
2. **보안**: 서버마다 다른 권한과 접근 범위를 Client 레벨에서 제어 가능
3. **확장성**: 새로운 서버를 추가할 때 기존 연결에 영향 없이 새 Client만 생성
4. **프로토콜 관리**: 각 Client가 서버와의 프로토콜 버전 협상을 독립적으로 처리

3) MCP 통신방식

- MCP는 **JSON-RPC 2.0** 기반으로 **stdio**와 **SSE**, **streamable-http** 3가지 통신 채널(transport)을 지원하는데, 이중 SSE 방식은 긴 연결을 유지하기에는 보안 위험이 있어서 폐기 예정이며, 대신 streamable-http 사용 권장
- JSON-RPC 2.0은 3가지 유형의 메시지로 구성

1. 요청(Request)

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "search_documents",
    "arguments": {
      "query": "매출 보고서"
    }
  }
}
```

- jsonrpc: 항상 "2.0"
- id: 요청을 식별하는 고유 값 (응답과 매칭)
- method: 호출할 메서드 이름
- params: 메서드에 전달할 매개변수

3) MCP 통신방식

2. 응답(Response)

<pre>{ "jsonrpc": "2.0", "id": 1, "result": { "content": [{ type: "text", text: "3건의 문서를 찾았습니다:\n- 2026년 3월 매출 보고서\n- 2026년 2월 매출 보고서\n- 2026년 1분기 매출 요약" }] } }</pre>	• id: 요청의 id와 동일한 값 • result: 성공 시 결과 데이터
--	---

3. 알림(Notification)

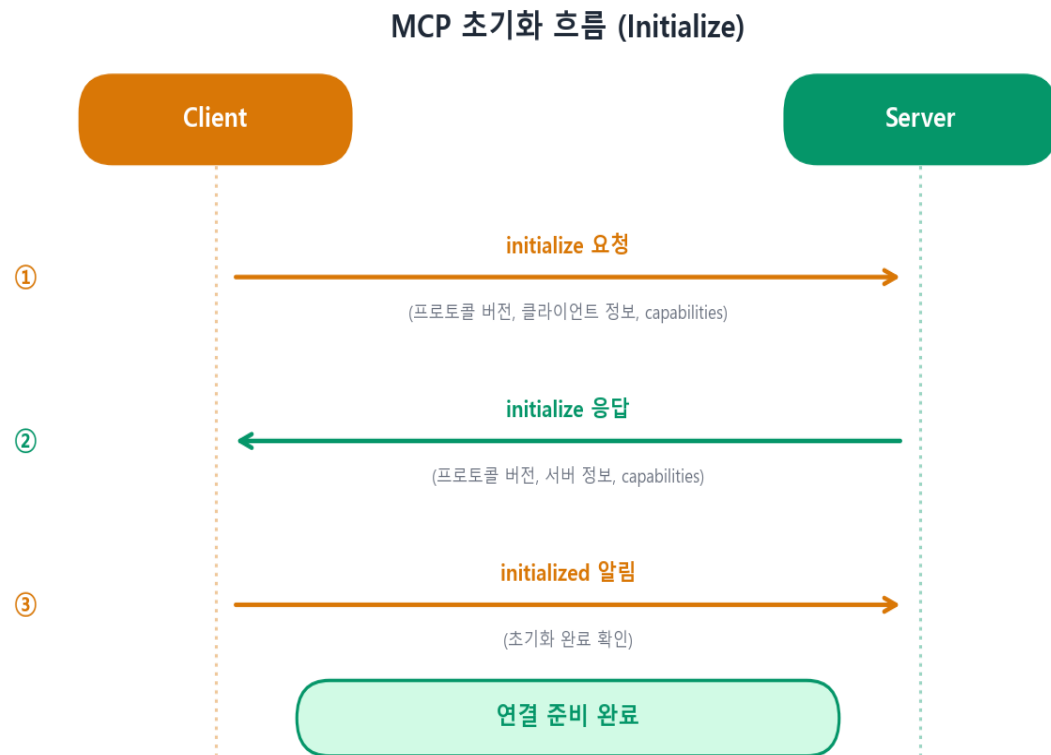
<pre>{ "jsonrpc": "2.0", "method": "notifications/tools/list_changed" }</pre>	• id가 없음 → 응답을 기대하지 않는 단방향 메시지 • 상태 변경이나 이벤트를 전달할 때 사용
---	--

3) MCP 통신방식

○ MCP 초기화 과정

- Client와 Server가 연결되면 반드시 초기화(Initialize) 과정을 거쳐야 하는데, 이과정을 통해 양측이 지원하는 기능(capabilities)을 교환

① Client → Server: initialize 요청



```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-03-26",
    "capabilities": {
      "roots": {
        "listChanged": true
      },
      "sampling": {}
    },
    "clientInfo": {
      "name": "Claude Desktop",
      "version": "1.5.0"
    }
  }
}
```

3) MCP 통신방식

② Server → Client: initialize 응답

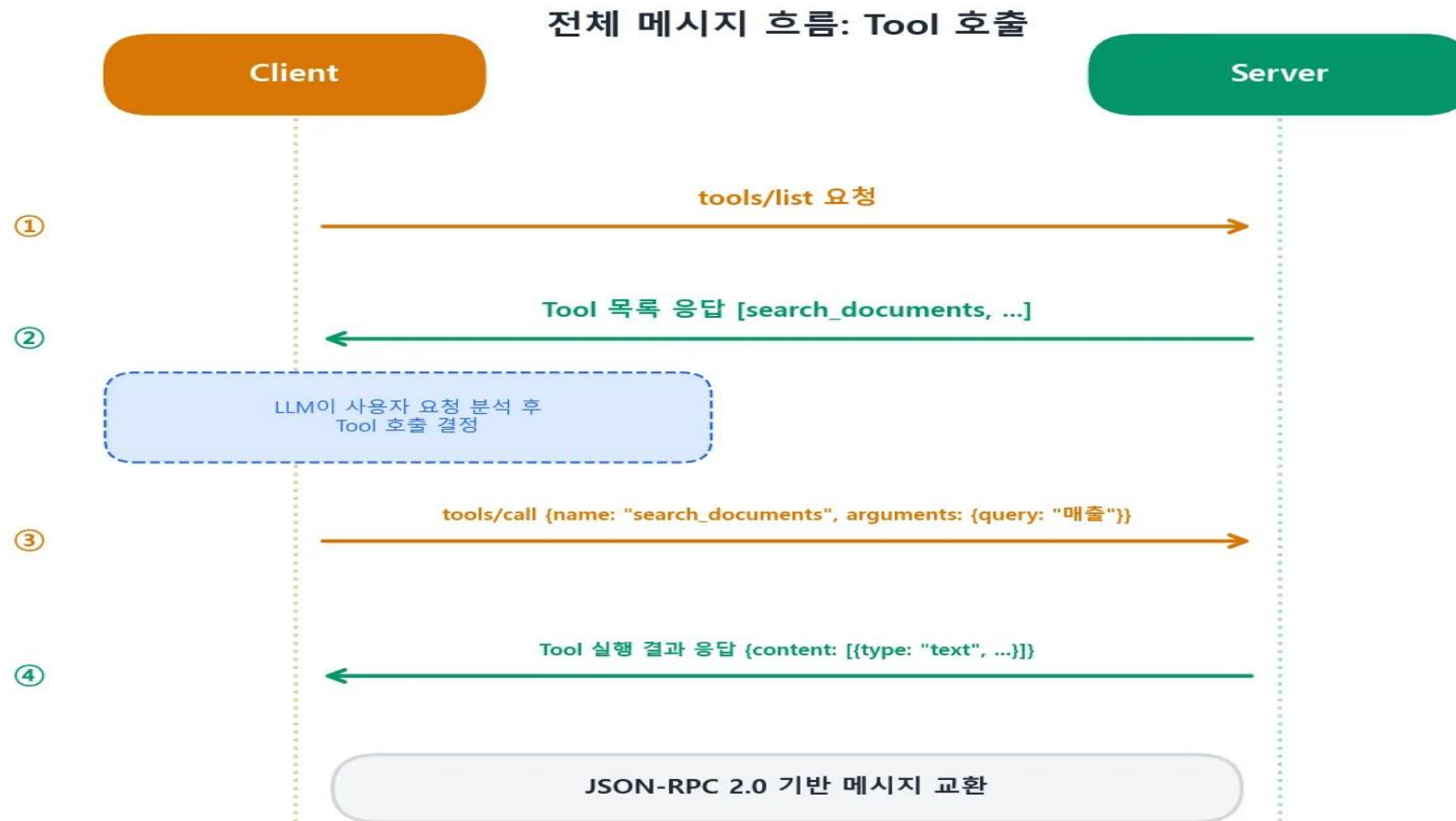
```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "protocolVersion": "2025-03-26",
    "capabilities": {
      "tools": {
        "listChanged": true
      },
      "resources": {
        "subscribe": true,
        "listChanged": true
      },
      "prompts": {
        "listChanged": true
      }
    },
    "serverInfo": {
      "name": "document-server",
      "version": "0.1.0"
    }
  }
}
```

③ Client → Server: initialized 알림

```
{
  "jsonrpc": "2.0",
  "method": "notifications/initialized"
}
```

3) MCP 통신방식

○ 초기화 이후 실제 Tool을 호출하는 전체 흐름 예제



3) MCP 통신방식

① tools/list 요청

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/list"
}
```

② tools/list 응답

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "tools": [
      {
        "name": "search_documents",
        "description": "문서 저장소에서 키워드로 문서를 검색합니다.",
        "inputSchema": {
          "type": "object",
          "properties": {
            "query": {
              "type": "string",
              "description": "검색할 키워드"
            }
          },
          "required": ["query"]
        }
      }
    ]
  }
}
```

③ tools/call 요청

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "method": "tools/call",
  "params": {
    "name": "search_documents",
    "arguments": {
      "query": "매출 보고서"
    }
  }
}
```

④ tools/call 응답

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "3건의 문서를 찾았습니다:\n- 2026년 3월 매출 보고서\n- 2026년 2월 매출 보고서\n- 2026년 1분기 매출 요약"
      }
    ],
    "isError": false
  }
}
```

3) MCP 통신방식

○ 에러 응답 형식: tool 실행 중 오류가 발생하면 두 가지 방식으로 에러 전달

(1) 프로토콜 수준 에러(JSON-RPC Error)

- MCP 프로토콜 자체의 문제 (잘못된 메서드, 잘못된 파라미터 등)

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "error": {
    "code": -32601,
    "message": "Method not found",
    "data": "unknown method: tools/invalid"
  }
}
```

- 주요 에러 코드:

코드	의미
-32700	Parse error (JSON 파싱 실패)
-32600	Invalid Request (잘못된 요청)
-32601	Method not found (존재하지 않는 메서드)
-32602	Invalid params (잘못된 파라미터)
-32603	Internal error (서버 내부 오류)

3) MCP 통신방식

(2) Tool 실행 수준 에러

- Tool은 정상적으로 호출되었지만, 실행 과정에서 비즈니스 로직 오류가 발생한 경우

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "검색 중 오류가 발생했습니다: 데이터베이스 연결 시간 초과"
      }
    ],
    "isError": true
  }
}
```

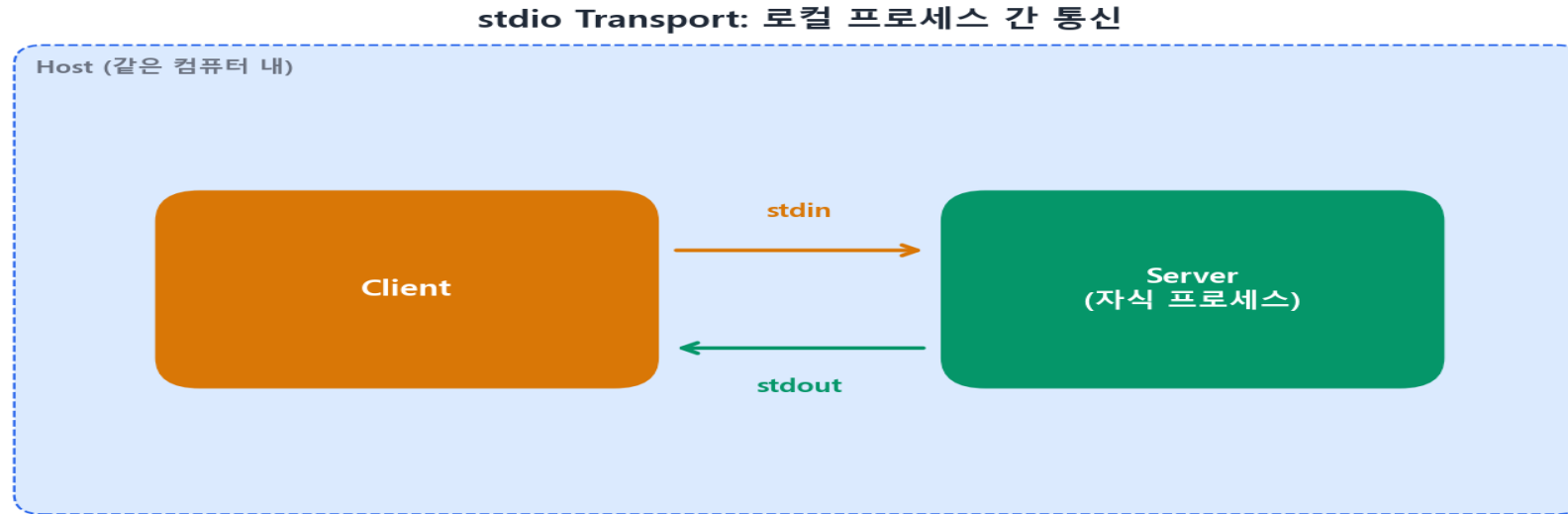
○ 프로토콜 수준 에러와 Tool 실행 수준 에러의 차이

- 프로토콜 에러는 통신 자체의 문제이고 Tool 에러는 기능은 호출되었지만 실행 중 문제가 발생한 경우에 해당
- HTTP로 비유하면, 프로토콜 에러는 404 Not Found이고 Tool 에러는 200 OK 이지만 응답 본문에 에러 메시지가 담긴 경우와 유사

3) MCP 통신방식

- MCP에서 Transport는 client와 Server 사이에서 JSON-RPC 메시지를 실제로 주고받는 통신 채널이며, MCP 2025-03-26 스펙은 공식 Transport로 **stdio**와 **Streamable HTTP**를 정의
- **Stdio(표준입출력) 방식: 로컬 프로세스 간 통신**
 - MCP 서버와 클라이언트가 표준 입출력(Standard Input/Output)을 통해 직접 데이터를 주고받는 통신 방식
 - MCP 서버는 명령 프롬프트나 터미널 환경에서 실행되며 클라이언트는 표준 입출력을 통해 서버와 직접 데이터를 주고 받음
 - 클라이언트는 stdin을 통해 데이터를 서버에 전송하고 서버는 stdout을 통해 처리 결과를 클라이언트에 반환
 - 클라이언트와 서버가 동일 컴퓨터에서 실행될 때 사용되며 로컬 테스트나 개발 중인 MCP 서버를 빠르게 테스트하는 용도로 적합

3) MCP 통신방식



- Stdio 방식의 장단점

장점	• 설정이 간단하고 별도 네트워크의 구성 없이 빠르게 테스트 가능 • 네트워크를 사용하지 않아 민감한 데이터가 외부로 노출될 위험이 적음 • 커서, 클로드 데스크탑 등 로컬 실행 기반의 도구와 연동하기에 적합
단점	• 클라이언트와 서버가 동일 시스템 또는 파일 경로 내에 있어야 하므로 원격 사용이 불가 • 병렬처리에 제약이 있으며 동시에 여러 세션을 관리하기 어려움 (Client:Server = 1:1 연결)

3) MCP 통신방식

- Claude Desktop에서 stdio 설정 예시:

```
{
  "mcpServers": {
    "my-server": {
      "command": "python",
      "args": ["/path/to/my_server.py"],
      "env": {
        "API_KEY": "your-key-here"
      }
    }
  }
}
```

위의 설정에서 Claude Desktop은 “python /path/to/my_server.py”을 자식 프로세스로 실행하고 stdin/stdout 으로 통신

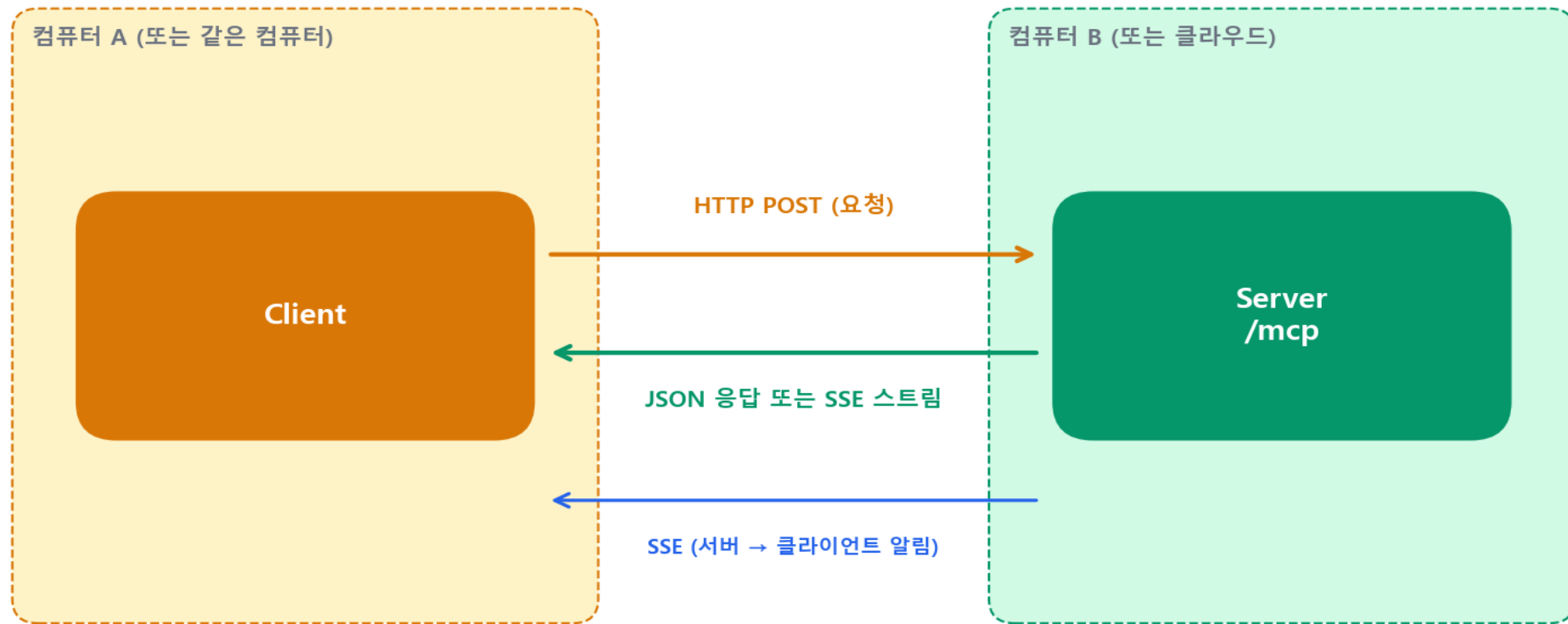
3) MCP 통신방식

○ Streamable HTTP 방식: 네트워크 기반 통신

- Streamable HTTP Transport는 HTTP를 기반으로 통신하는데 서버가 HTTP 엔드포인트를 제공하고, 클라이언트가 HTTP 요청으로 메시지를 보냄
- MCP 2025-03-26 스펙에서 이전의 HTTP+SSE Transport를 대체하여 도입
- 동작 방식:
 1. 서버가 HTTP 엔드포인트(기본: /mcp)를 제공
 2. 클라이언트가 JSON-RPC 메시지를 HTTP POST로 전송
 3. 서버는 응답을 JSON으로 직접 반환하거나, SSE(Server-Sent Events) 스트림으로 반환
 4. 서버가 클라이언트에게 알림을 보내야 할 때는 SSE를 사용

3) MCP 통신방식

Streamable HTTP Transport: 네트워크 기반 통신



3) MCP 통신방식

- 세션 관리: Streamable HTTP는 MCP-Session-Id 헤더로 세션을 관리

```
# 초기화 요청
POST /mcp HTTP/1.1
Content-Type: application/json

{"jsonrpc": "2.0", "id": 1, "method": "initialize", ...}

# 초기화 응답 (세션 ID 발급)
HTTP/1.1 200 OK
Mcp-Session-Id: abc123-def456
Content-Type: application/json

{"jsonrpc": "2.0", "id": 1, "result": {...}}

# 이후 요청에 세션 ID 포함
POST /mcp HTTP/1.1
Mcp-Session-Id: abc123-def456
Content-Type: application/json

{"jsonrpc": "2.0", "id": 2, "method": "tools/list"}
```

3) MCP 통신방식

- Streamable HTTP의 장단점

장점	• 원격 통신: 네트워크를 통해 다른 컴퓨터의 서버에 연결 가능 • 확장 가능: 로드 밸런서, 리버스 프록시 등 기존 인프라와 호환 • 다중 클라이언트: 하나의 서버가 여러 클라이언트를 동시에 서비스 가능 • 기존 인프라 활용: 기존의 HTTP 기반 인증, 모니터링, 방화벽 규칙을 그대로 사용 가능 • 클라우드 배포: 서버를 클라우드에 배포하여 어디서든 접근 가능
단점	• 복잡성: 네트워크 설정, HTTPS, 인증 등 추가 구성이 필요 • 지연: 네트워크 지연(latency) 발생 • 보안 고려: HTTP 엔드포인트가 외부에 노출되므로 인증/인가 구현이 필수

3) MCP 통신방식

○ SSE(Server-Sent Events) 방식: 폐기 예정(Deprecated)

- MCP 서버가 HTTP 기반의 스트리밍 통신을 통해 클라이언트에게 실시간으로 데이터를 전송하는 방식
- 이전 MCP 스펙(2024-11-05)에서 HTTP+SSE Transport가 사용되었으며, 두 개의 엔드포인트를 사용
 - GET /sse: 서버에서 클라이언트로의 SSE 스트림 (서버 → 클라이언트)
 - POST /messages: 클라이언트에서 서버로의 메시지 (클라이언트 → 서버)
- 2025-03-26 스펙에서 Streamable HTTP로 통합되었으며, 기존 SSE방식은 폐기 예정이나 하위 호환성을 위해 일부 서버에서는 지원 중이나 향후 완전 제거될 수 있음

3) MCP 통신방식

○ 정리

- MCP의 설계철학은 “같은 서버 코드를 Transport만 바꿔서 다른 환경에 배포할 수 있다”는 것이므로 서버의 비즈니스 로직(tool, Resource, Prompt)은 그대로 유지하면서 Transport 계층만 stdio에서 Streamable HTTP로 교체 가능
- Stdio와 Streamable HTTP 사용 기준

기준	stdio	Streamable HTTP
서버 위치	로컬 (같은 컴퓨터)	로컬 또는 원격
사용자 수	1명 (단일 사용자)	1명 또는 다수
네트워크 필요	불필요	필요 (HTTP 통신)
설정 난이도	매우 쉬움	중간~높음
배포 환경	개발, 개인 사용	프로덕션, 팀 공유, 클라우드
보안 구현	최소 (OS 수준)	필수 (인증, HTTPS 등)
성능	매우 빠름	네트워크 의존
적합한 경우	개인 도구, 로컬 파일 처리, 개발/테스트	SaaS 서비스, 팀 공유 도구, 프로덕션

3) MCP 통신방식

○ 실무 가이드라인:

- **개발 단계 / 개인 사용:** stdio로 시작하세요. 설정이 간단하고, Claude Desktop에서 바로 테스트할 수 있습니다.
- **팀 공유 / 프로덕션:** Streamable HTTP로 전환하세요. 중앙 서버에 배포하여 팀 전체가 사용할 수 있습니다.
- **하이브리드:** 개발은 stdio로, 배포는 Streamable HTTP로 하는 것이 일반적인 패턴입니다.

개발 흐름: stdio에서 Streamable HTTP로



4) MCP의 구성요소 : Tools, Resource, Prompt

(1) 도구(tools): AI가 “행동”하게 만드는 함수

- MCP서버가 외부에 노출하는 실행가능한 함수로 사용자가 요청한 작업을 실제로 실행하는 역할을 수행
- AI가 행동을 수행하는 핵심 매커니즘
- Tool의 구성요소
 - 이름(name): Tool을 식별하는 고유한 문자열 (예: search_documents)
 - 설명(description): LLM이 이 Tool의 용도를 이해하기 위한 자연어 설명
 - 입력 스키마(inputSchema): JSON Schema 형식으로 정의된 입력 파라미터
 - 출력: 실행 결과 (텍스트, 이미지, 또는 다른 콘텐츠)
- Tool의 입력 파라미터는 JSON Schema로 정의되며, 이를 통해 LLM은 어떤 값을 전달하여야 하는지 정확한 파악이 가능해 짐

4) MCP의 구성요소 : Tools, Resource, Prompt

- 이 스키마를 보고 LLM은 다음과 같은 판단이 가능

- query는 필수 입력이고, 문자열로 검색어를 전달해야 한다
- max_results는 선택적이며, 정수로 결과 수를 지정할 수 있다
- category는 선택적이며, 정해진 값 중 하나를 선택할 수 있다

```
{
  "name": "search_documents",
  "description": "문서 저장소에서 키워드로 문서를 검색합니다. 검색어와 최대 결과 수를 지정할 수 있습니다.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "query": {
        "type": "string",
        "description": "검색할 키워드 또는 문장"
      },
      "max_results": {
        "type": "integer",
        "description": "반환할 최대 결과 수 (기본값: 10)",
        "default": 10
      },
      "category": {
        "type": "string",
        "description": "문서 카테고리 필터",
        "enum": ["all", "report", "memo", "manual"]
      }
    },
    "required": ["query"]
  }
}
```

4) MCP의 구성요소 : Tools, Resource, Prompt

○ Tool Annotation

- MCP 2025-03-26 스펙에서는 Tool에 Annotation 추가 가능
- Annotation은 Tool의 동작 특성을 Host에게 알려주는 메타 데이터

```
{
  "name": "delete_file",
  "description": "지정된 경로의 파일을 삭제합니다.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "path": {
        "type": "string",
        "description": "삭제할 파일의 경로"
      }
    }
  },
  "required": ["path"]
},
"annotations": {
  "title": "파일 삭제",
  "readOnlyHint": false,
  "destructiveHint": true,
  "idempotentHint": true,
  "openWorldHint": false
}
}
```

Annotation	의미	예시
title	사용자에게 표시할 이름	"파일 삭제"
readOnlyHint	읽기 전용 여부 (서버 상태를 변경하지 않음)	검색 Tool → true
destructiveHint	파괴적 작업 여부 (되돌리기 어려운 변경)	삭제 Tool → true
idempotentHint	멱등성 여부 (같은 입력으로 반복 호출해도 결과 동일)	조회 Tool → true
openWorldHint	외부 세계에 영향 여부 (외부 API 호출 등)	이메일 발송 → true

* Annotation의 **Hint 접미사에 주목**하세요. 이 값들은 강제 규칙이 아니라 "힌트"입니다. Host는 이 정보를 참고하여 사용자에게 확인을 요청하거나, UI에 경고를 표시하는 등의 결정을 내립니다. 예를 들어, destructiveHint: true인 Tool을 호출할 때 "이 작업은 되돌릴 수 없습니다. 계속하시겠습니까?"라는 확인 대화상자를 보여줄 수 있습니다.

4) MCP의 구성요소 : Tools, Resource, Prompt

○ MCP에서 Tool 호출 흐름

- 사용자가 "최근 매출 보고서를 찾아줘"라고 요청했을 때, Tool 호출의 단계별 실행과정



4) MCP의 구성요소 : Tools, Resource, Prompt

- 이 흐름에서 중요한 점은 LLM이 Tool 호출 여부를 결정한다는 것임. Host는 LLM에게 사용 가능한 Tool 목록을 알려주고 LLM은 사용자의 요청을 분석하여 어떤 Tool을 어떤 파라미터로 호출할지 판단함

- MCP에서는 @mcp.tool()로 등록하여 사용
- 사용 예:

```
from mcp import FastMCP
mcp = FastMCP("add")
@mcp.tool()
def add(a: int, b: int) -> int:
    """a와 b를 더하기"""
    return a + b
if __name__ == "__main__":
    mcp.run()
```

4) MCP의 구성요소 : Tools, Resource, Prompt

(2) 리소스(Resource): AI가 “참조”할 수 있는 데이터

- LLM이 참고할 수 있도록 문맥이나 데이터를 사전에 제공하는 역할 수행
- Resource는 MCP 서버가 외부에 노출하는 읽기 전용 데이터
- Tool이 “행동(실행)”을 위한 것이라면, Resource는 “참조(읽기)” 위한 것임
- Resource는 URI(Uniform Resource Identifier)로 식별
- MCP에서는 @mcp.resource()를 사용해 리소스를 등록하고 활용
- 사용 예:

```
@mcp.resource("user_profile")
def get_user_profile() -> dict:
    return {
        "name": "홍길동",
        "role": "마케팅 매니저",
        "location": "대전"
    }
```

```
file:///home/user/documents/report.pdf
db://sales/monthly_summary
github://repo/main/README.md
config://app/settings
```

4) MCP의 구성요소 : Tools, Resource, Prompt

- Tool과 Resource의 차이

구분	Tool	Resource
목적	행동 수행 (실행)	데이터 참조 (읽기)
비유	"검색해줘", "보내줘"	"이 파일 보여줘", "이 데이터 읽어줘"
상태 변경	가능 (DB 수정, 메일 발송 등)	없음 (읽기 전용)
호출 주체	LLM이 판단하여 호출	사용자 또는 애플리케이션이 선택
식별 방법	이름(name)	URI
위험도	높을 수 있음 (파괴적 작업 가능)	낮음 (읽기만)

❖ Tool과 Resource의 가장 큰 차이점은 호출 주체임. Tool은 LLM이 자동으로 호출 여부를 결정하지만, Resource는 일반적으로 사용자나 애플리케이션이 명시적으로 선택함. 보안 관점에서 볼 때 자동 실행되는 것(tool)과 명시적으로 참조하는 것(resource)은 위험 수준이 다름

4) MCP의 구성요소 : Tools, Resource, Prompt

○ Resource의 두 가지 유형

1. 직접 리소스 (Direct Resource)

- 고정된 URI로 특정 데이터를 노출함

```
{
  "uri": "config://app/database",
  "name": "데이터베이스 설정",
  "description": "현재 애플리케이션의 데이터베이스 연결 설정",
  "mimeType": "application/json"
}
```

2. 리소스 템플릿 (Resource Tamplate)

- URI 템플릿으로 동적 리소스를 정의함. 클라이언트가 템플릿의 변수를 채워 구체적인 리소스를 요청

```
{
  "uriTemplate": "db://sales/{year}/{month}",
  "name": "월별 매출 데이터",
  "description": "지정된 연도와 월의 매출 데이터를 반환합니다.",
  "mimeType": "application/json"
}
```

❖ 이 템플릿을 사용하면 db://sales/2026/05처럼 구체적인 URI로 데이터를 요청할 수 있음

4) MCP의 구성요소 : Tools, Resource, Prompt

(3) 프롬프트(Prompt): 서버가 제안하는 대화 템플릿

- LLM에게 주어지는 지시문 또는 입력 문장으로 사용자의 요청을 일관된 방식으로 해석하고 처리할 수 있도록 지원하는 기능 수행
- Tool이 “AI가 호출하는 함수”라면 Prompt는 “사용자가 선택하는 대화 시나리오”임
- Prompt의 구조
 - 이름(name): Prompt를 식별하는 고유 문자열
 - 설명(description): 사용자가 이 Prompt의 용도를 이해하기 위한 설명
 - 인자(arguments): Prompt를 커스터마이징하기 위한 매개변수
 - 메시지(messages): 실제 대화 내용을 구성하는 메시지 배열

4) MCP의 구성요소 : Tools, Resource, Prompt

```
{
  "name": "monthly_closing",
  "description": "월말 결산 점검을 위한 체크리스트를 생성합니다.",
  "arguments": [
    {
      "name": "year",
      "description": "결산 연도",
      "required": true
    },
    {
      "name": "month",
      "description": "결산 월",
      "required": true
    },
    {
      "name": "department",
      "description": "부서명 (선택사항)",
      "required": false
    }
  ]
}
```

4) MCP의 구성요소 : Tools, Resource, Prompt

- Prompt가 반환하는 메시지

- 사용자가 prompt를 선택하고 인자를 입력하면, 서버는 미리 구성된 메시지 배열을 반환함

```
{
  "messages": [
    {
      "role": "user",
      "content": {
        "type": "text",
        "text": "2026년 3월 영업부의 월말 결산 점검을 시작합니다.\n\n다음 항목을 순서대로 확인해주세요:\n1. 매출 데이터 정합성 확인\n2. 미수금 현황 점검\n3. 비용 처리 완료 여부\n4. 예산 대비 실적 분석\n5. 이상 거래 탐지\n\n각 항목에 대해 상태와 조치사항을 정리해주세요."
      }
    }
  ]
}
```

- * 이 메시지는 대화의 시작점으로 Host는 이 메시지를 대화에 삽입하고 LLM이 이에 따라 응답을 생성함. Prompt에서 참조하는 Resource를 함께 삽입하거나, 특정 Tool 사용을 유도하는 것도 가능

4) MCP의 구성요소 : Tools, Resource, Prompt

- Prompt의 실무 활용: 반복 업무의 표준화

활용 시나리오	Prompt 예시	효과
코드 리뷰	code_review(language, file)	리뷰 기준 일관성 확보
일일 보고	daily_report(date, team)	보고서 형식 통일
장애 분석	incident_analysis(service, time)	분석 절차 표준화
데이터 검증	data_validation(table, rules)	검증 기준 공유
온보딩 가이드	onboarding(role, team)	신규 입사자 교육 표준화

- MCP에서는 @mcp.prompt() 또는 add_prompt()를 사용해 프롬프트를 등록

- 사용 예:

```
@mcp.prompt()
def review_code(code: str) -> str:
    return f"Please review this code:\\n\\n{code}"
```

4) MCP의 구성요소 : Tools, Resource, Prompt

○ 도구, 리소스, 프롬프트(Prompt) 비교

구성요소	설명	실행방식	예시
도구	실행 가능한 함수	LLM이 직접 호출	계산, API 요청
리소스	참조용 데이터 또는 문맥 정보	클라이언트가 LLM 프롬프트에 포함하여 전달	회사소개, 가격표
프롬프트	입력형식이나 LLM에게 요청할 의도를 담은 예시 문장 또는 템플릿	LLM에게 어떤 방식으로 응답할지 방향을 지정함	요약해줘, ~ 스타일로 바꿔줘, 키워드만 뽑아줘 등

출처: 서지영, 나만의 MCP 서버만들기 with 커서 AI (2025, 길벗출판사)

5) Sampling: 서버가 LLM에게 되묻는 역방향 호출

○ Sampling의 개념

- 지금까지의 작업 흐름은 사용자가 요청하면 Host가 서버에게 Tool을 호출하고 결과를 받는 구조(Host → Server 방향)
- 반면 Sampling은 Server → Host 방향으로, 서버가 LLM의 추론 능력을 빌려쓰는 매커니즘

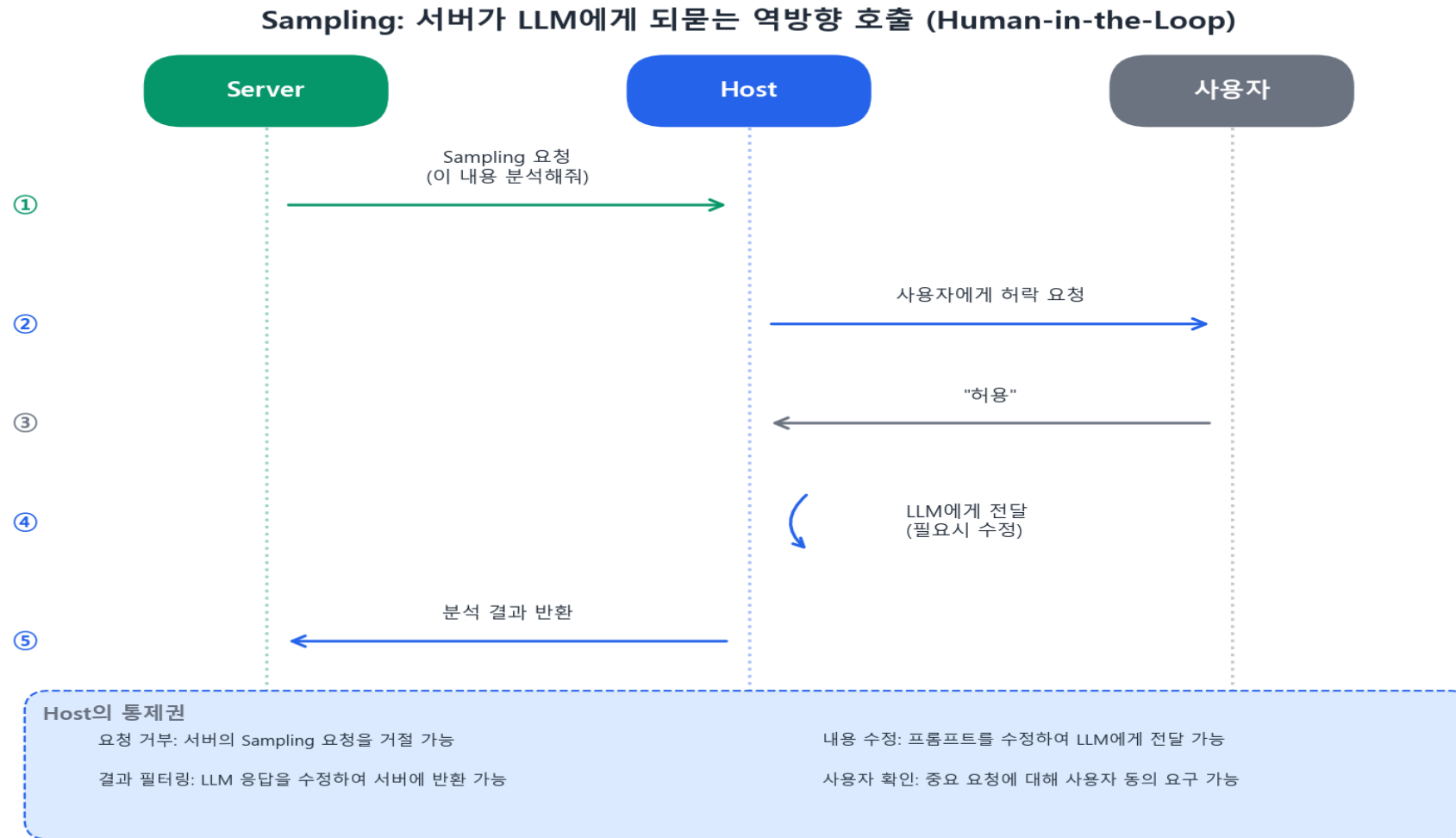
○ 서버가 LLM에게 다시 묻는 이유

- 서버가 tool을 처리하는 도중, LLM의 판단력이 필요한 경우 등
- 예시: 자동 코드 리뷰 서버
 1. 사용자가 "이 PR을 리뷰해줘"라고 요청
 2. Host가 review_pull_request Tool을 호출
 3. 서버가 GitHub에서 PR의 변경 사항을 가져옴
 4. **여기서 문제:** 서버는 코드 변경 사항은 가져왔지만, "이 코드가 좋은지 나쁜지" 판단하는 건 LLM의 영역
 5. 서버가 Sampling을 통해 Host에게 "이 코드 변경 사항을 분석해줘"라고 요청
 6. Host의 LLM이 분석 결과를 반환
 7. 서버가 분석 결과를 정리하여 최종 결과를 Host에게 반환

○ Host가 중간에서 통제 (Human-in-the-loop)

- Sampling에서 가장 중요한 원칙은 Host가 완전한 통제권을 가진다는 점

5) Sampling: 서버가 LLM에게 되묻는 역방향 호출



5) Sampling: 서버가 LLM에게 되묻는 역방향 호출

- Host는 Sampling 요청에 대해 다음과 같은 통제 수행이 가능
 - 요청 거부: 서버의 Sampling 요청을 거절할 수 있습니다.
 - 내용 수정: 서버가 보낸 프롬프트를 수정하여 LLM에게 전달할 수 있습니다.
 - 결과 필터링: LLM의 응답을 수정하여 서버에게 반환할 수 있습니다.
 - 사용자 확인: 중요한 요청에 대해 사용자에게 동의를 구할 수 있습니다.
- Sampling은 강력하지만 Host의 완전한 통제 하에서 동작함. 서버가 마음대로 LLM을 호출할 수 없고 반드시 Host의 허가를 받아야만 함. 이는 보안과 비용 관리 측면에서 매우 중요한 설계 원칙임

6) Roots와 Elicitation

(1) Roots: 서버에게 알려주는 작업 범위

- Roots는 클라이언트가 서버에게 “이 범위 안에서 작업해”라고 알려주는 메커니즘
- 예를 들어, 파일 시스템 MCP 서버가 있다고 할 때, 이 서버에게 아무런 제한없이 파일 접근을 허용하면 보안 상 문제 발생 소지가 있음. 이때 Roots를 사용하면 특정 디렉터리만 접근 가능하도록 범위를 지정할 수 있음

```
{
  "roots": [
    {
      "uri": "file:///home/user/projects/my-app",
      "name": "내 프로젝트"
    },
    {
      "uri": "file:///home/user/documents",
      "name": "문서 폴더"
    }
  ]
}
```

6) Roots와 Elicitation

- Roots의 특성

- 클라이언트가 서버에게 **제안하는** 작업 범위입니다 (강제가 아닌 가이드).
- 서버는 Roots 정보를 참고하여 관련 있는 데이터만 제공할 수 있습니다.
- 런타임 중 Roots가 변경되면 클라이언트가 `notifications/roots/list_changed`를 서버에게 알립니다.

- 단 Roots는 보안 메커니즘이 아닌 정보제공 메커니즘이기 때문에 서버가 Roots 범위 밖의 데이터에 접근하는 것을 기술적으로 막지는 않음

(2) Elicitation: 서버가 사용자에게 추가 입력을 요청

- Elicitation은 MCP 2025-03-26 스펙에서 추가된 기능으로, 서버가 Tool 실행 도중 사용자에게 직접 추가 정보를 요청하는 메커니즘임

6) Roots와 Elicitation

- Sampling과 Elicitation의 차이점

구분	Sampling	Elicitation
대상	LLM에게 요청	사용자에게 요청
목적	LLM의 추론 능력 활용	사용자의 결정/입력 필요
예시	"이 코드 분석해줘"	"어느 브랜치에 배포할까요?"

- 활용 사례: 서버가 배포 Tool을 실행하는 중에 배포 대상 환경을 사용자에게 물어야 하는 상황이라면:

```
{
  "method": "elicitation/create",
  "params": {
    "message": "배포할 환경을 선택해주세요.",
    "requestedSchema": {
      "type": "object",
      "properties": {
        "environment": {
          "type": "string",
          "description": "배포 환경",
          "enum": ["development", "staging", "production"]
        },
        "confirm": {
          "type": "boolean",
          "description": "배포를 진행하시겠습니까?"
        }
      },
      "required": ["environment", "confirm"]
    }
  }
}
```

* Host는 요청을 받아 사용자에게 UI(대화상자, 드롭다운 등)를 통해 입력을 받고, 결과를 서버에 반환

6) Roots와 Elicitation

- Elicitation의 핵심 원칙:

- Host가 Elicitation 요청을 **거부**할 수 있습니다 (capabilities에서 지원 여부 선언).
- 사용자의 입력은 requestedSchema에 정의된 JSON Schema로 검증됩니다.
- 사용자가 **취소**하면 서버에 "dismissed" 결과가 반환됩니다.

7) MCP의 주요 장점

- 개방성을 통한 상호 운용성 향상: MCP로 구축된 애플리케이션은 호환되는 모든 도구 및 데이터 소스와 잘 작동한다는 장점을 지님
- 구현 후 재사용 용이: 한번 구현한 이후 생태계 전체에서 재사용할 수 있다는 장점 제공
- 생산성 향상: 특정 작업을 수행하는데 기 구축된 MCP 생태계의 MCP 서버를 활용함으로써 생산성 향상 가능

8) MCP의 기능적 한계 및 보안 취약성

○ MCP의 기능적 한계

- MCP는 LLM이 외부도구와 구조화된 방식으로 상호작용할 수 있도록 설계된 도구 호출을 위한 표준화된 통신규격으로 기본적으로 도구 단위의 단일 호출에 최적화되어있음. 따라서 도구간의 순차 실행, 조건 분기, 결과 공유와 같은 복잡한 워크 플로우로 구성하기에는 한계가 있음
- 예를 들면, 도구 A의 출력결과를 도구 B의 입력으로 넘기거나 여러 도구를 조합해 하나의 작업을 완성하려면 개발자가 별도의 로직이나 중간처리를 직접 구현해야 함
- MCP에서는 각 도구가 독립적으로 정의되기 때문에 입출력 스펙이 일치하지 않을 경우 호출 실패나 잘못된 결과가 발생할 수 있음
- 예를 들면, 입력값 타입이 정의되지 않았거나 출력 형식에 대한 설명이 부족할 경우 LLM이 도구를 인식하지 못하거나 오작동할 수 있음
- 전송방식에 있어서도 제약이 있는데, Stdio는 로컬 환경에서는 유용하지만 웹, 모바일 환경에는 부적합하고 SSE 방식은 서버에서 클라이언트로 데이터를 지속적으로 푸시하는 단방향 스트리밍 방식이기 때문에 클라이언트에서 서버로 실시간 데이터를 전송해야 하는 경우는 부적합함

8) MCP의 기능적 한계 및 보안 취약성

○ MCP의 보안 취약성

(1) 외부 접근 용이성에 따른 취약성

- MCP 서버는 /messages/ 같은 엔드포인트를 외부에 노출하는 구조이므로 인증없이 접근이 가능할 경우 보안상 심각한 문제 발생 소지가 있음
- 즉 누구나 엔드포인트에 접근해 도구를 무단 호출할 수 있으며, 시스템 명령어 실행 도구가 노출된 경우, 치명적 보안사고 발생 위험이 존재

(2) 사용자 입력 검증 부족

- 도구가 사용자 입력을 별도의 필터링 없이 처리할 경우 명령어 주입, 파일 시스템 접근, 악성 코드 실행 등의 위험에 노출 될 소지가 많음

(3) 도구간 인증 및 권한 관리 기능 부족

- 여러 MCP 서버와 도구가 연결된 복합 환경에서 도구간 상호 인증 및 신뢰기반 설정이 부족할 경우 중간자 공격(Man-In-The-Middle Attack, MITM), 도구간 무단 호출 등의 문제가 발생할 수 있어 전체 시스템의 보안 취약점으로 이어질 가능성이 높음

(4) 민감한 로그 정보의 노출 위험

- MCP는 요청과 응답 데이터를 로그로 기록하는 구조를 갖고 있기 때문에 별도의 필터링 없이 로그를 저장할 경우 사용자 개인정보, 시스템 내부 정보 등 민감 데이터가 노출될 위험이 있음

□ MCP 이론 핵심 내용 요약

구성 요소	역할	핵심 포인트
Host	사용자 인터페이스 + LLM	Client 관리, 보안 정책 적용
Client	MCP 서버 커넥터	Server와 1:1 통신
Server	외부 시스템 래퍼	Tool/Resource/Prompt 노출
Tool	실행 가능한 함수	AI가 "행동"하는 메커니즘
Resource	읽기 전용 데이터	URI로 식별, 참조용
Prompt	대화 템플릿	반복 업무 표준화
Sampling	역방향 LLM 호출	서버가 LLM 추론 활용
Roots	작업 범위 지정	보안 가이드
Elicitation	사용자 입력 요청	추가 정보 수집
JSON-RPC 2.0	메시지 형식	요청/응답/알림
stdio	로컬 Transport	빠르고 간단
Streamable HTTP	네트워크 Transport	원격, 확장 가능

1-2 MCP 관련 용어 (클로드 중심)

1) 스킬(Skill)

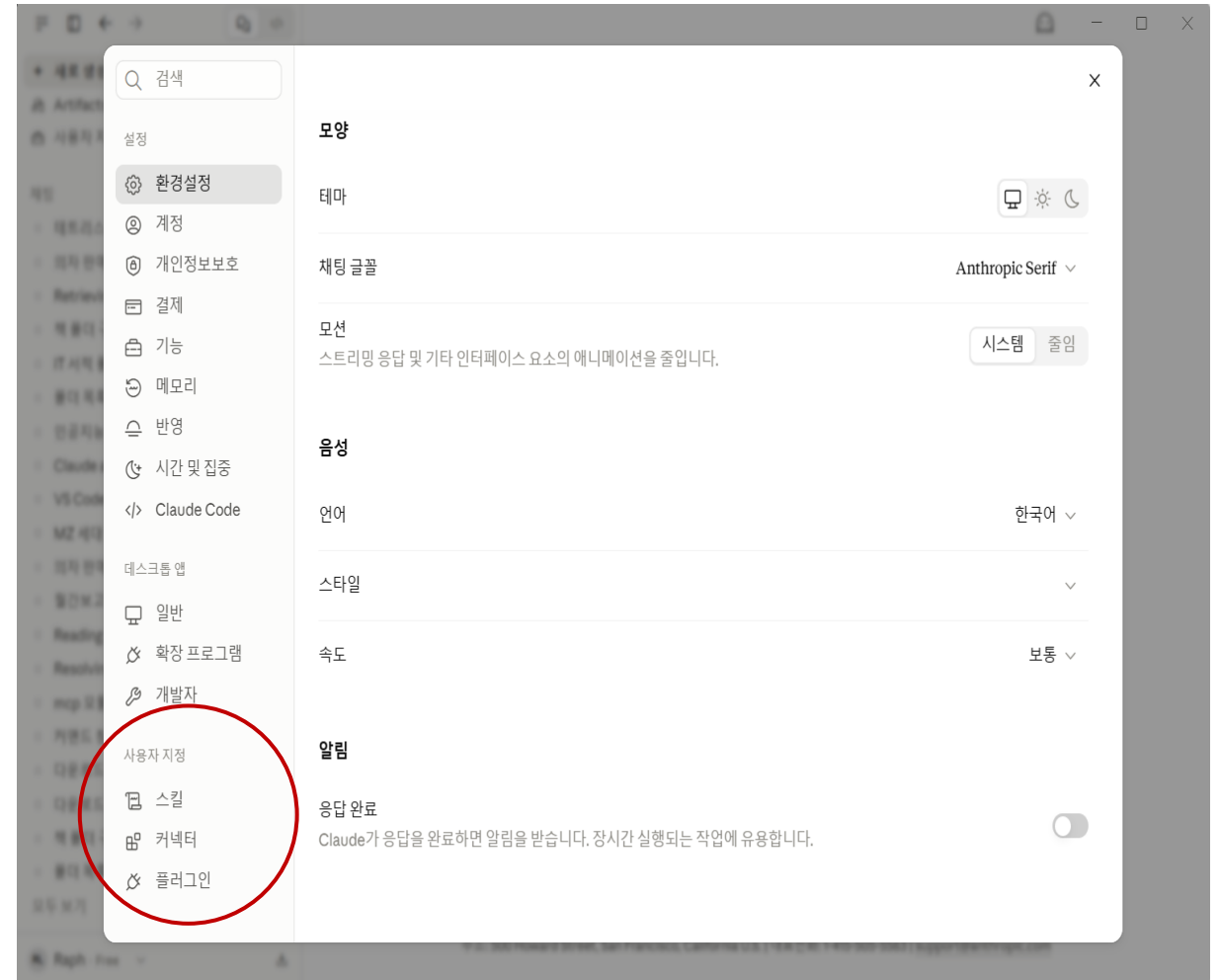
- AI가 특정 업무절차나 표준지침(SKILL.md)을 따르도록 만든 지식 패키지
- 반복작업을 자동화하고 일관된 결과물을 얻는데 사용

2) 커넥터(Connector)

- 클로드가 MCP 표준을 기반으로 외부 앱, 업무 도구, 데이터 소스 등 다양한 소프트웨어와 연동하여 실시간으로 데이터를 읽고 작업을 수행할 수 있도록 지원하는 기능

3) 플러그인(Plugin)

- 스킬 + MCP 서버 + 후크(hook)
- [클로드코드 플러그인 가이드 2026](#)



2. MCP 실습 환경 설정

□ 주요 설치 프로그램

1) Node.js

- Node.js란? 크롬 V8 자바스크립트 엔진으로 빌드된 자바스크립트 런타임으로, **이벤트 기반, 논블로킹 I/O** 모델을 사용하여 가볍고 빠르며 확장 가능한 네트워크 프로그램을 쉽게 작성할 수 있도록 함. node.js 패키지 생태계인 npm(Node Package Manager)은 세계적으로 유명한 오픈소스 라이브러리 생태계의 하나임
- 2008년 9월 구글이 크롬 웹브라우저에 이식한 자바스크립트 엔진 V8을 오픈소스로 공개, 2009년 유럽 JSConf의 라이언 달(Ryan Dahl)이 V8을 이용하여 Node.js 발표
- <https://nodejs.org/ko/download> 접속
- Windows 설치프로그램(.msi) 선택

노드의 장단점

장점	단점
멀티스레드 방식에 비해 컴퓨터 자원을 적게 사용함	싱글스레드라서 CPU 코어를 하나만 사용
I/O 작업이 많은 서버로 적합	CPU 작업이 많은 서버로는 부적합
멀티스레드 방식보다 쉬움	하나뿐인 스레드가 멈추지 않도록 관리해야 함
웹 서버가 내장되어 있음	서버 규모가 커졌을 때 서버를 관리하기 어려움
자바스크립트를 사용함	어중간한 성능
JSON 형식과 호환하기 쉬움	

출처: 조현영, Node.js 교과서, 도서출판 길벗, 2018

잠깐!! Core와 Thread

○ 코어와 스레드의 차이

구분	코어(Core)	스레드(Thread)
정의	하드웨어적인 물리적 처리 장치	소프트웨어적인 논리적 처리 흐름
비유	주방의 요리사 인원수	요리사가 작업하는 손 (일거리 처리 통로)
성능	많을수록 고사양 작업(게임, 3D 랜더링 등) 가능	많을수록 다중 작업(스트리밍, 창 여러 개 띄우기 등)이 매끄러움
예시	4 Core 8Thread CPU이라면, 운영체제는 4개의 코어가 한번은 스레드 TH 0, TH 2, TH4, TH 6으로 동작했다가, 다음엔 스레드 TH 1, TH 3, TH5, TH 7으로 콘텍스트 스위칭 (Context Switching)하면서 동작하는 방식	



KISTI 슈퍼컴퓨터 5호기 누리온(NURION)

시스템 개요 및 구성 | 누리온

docs-ksc.gitbook.io/nurion-user-guide/undefined/nurion-system-overview-and-configuration

누리온 지침서

초보사용자 가이드 | 누리온 지침서 | 누리온 지침서 | 활용정보 | MyKSC 지침서

누리온 지침서

1 시스템

시스템 개요 및 구성

사용자 환경

사용자 프로그래밍 환경

스케줄러(PBS)를 통한 작업 실행

사용자 지원

2 소프트웨어

ANSYS FLUENT

ANSYS CFX

Abaqus (2023 버전 이전)

Abaqus (2024 버전 이후)

가우시안16(Gaussian16) LINDA

가우시안16(Gaussian16)

3 부록

작업 스크립트 주요 키워드

Conda

Singularity 컨테이너

Lustre stripe

데이터 아카이빙

MVAPICH2 성능 최적화 옵션

딥러닝 프레임워크 병렬화

공통라이브러리 목록

데스크톱 가상화(VDI)

버스트버퍼(Burst Buffer)

플랫 노드(Flat node)

DTN(데이터전송노드)

GitBook 제공

Main System

Compute Nodes(8,305 ea)

CPU Nodes(132 ea)

Intel Omni Path High-Speed Interconnect

Burst Buffer

0.8PB

1.26PB

20.3PB

10PB

Management(2)

Scheduler(2)

Cloud Agent & Storage(1)

Cloud Agent(2)

Log(4)

Kernel Dump(1)

Multi-Port(2)

Video(2)

[누리온 구성도]

구분	KNL 계산노드	Cpu-only 노드
제조사 및 모델	Cray CS500	
노드수	8,305	132
성능최고성능(Rpeak)	25.3 PFlops	0.4 PFlops
프로세서	모델	Intel Xeon Phi 7250(KNL)
	CPU당 이론성능	3.0464 TFLOPS
	CPU당 코어수	68
	노드당 CPU수	1
메인메모리	On-package Memory	16GB, 490GB/s
	모델	16GB DDR4-2400
	구성	16GB x6, 6Ch per CPU
	노드 당 메모리(GB)	96GB
전체크기	CPU당 대역폭	115.2GB/s
	전체크기	778.6TB
토폴로지	Fat Tree	

이 페이지에서

가. 개요

나. 계산 노드

- KNL(메니코어) 노드
- SKL(CPU-only) 노드

다. 인터커넥트 네트워크

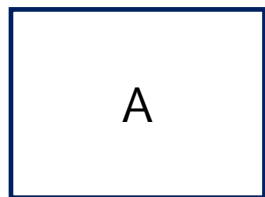
라. 스토리지

- 병렬파일시스템 및 버스트버퍼 구성
- 버스트버퍼

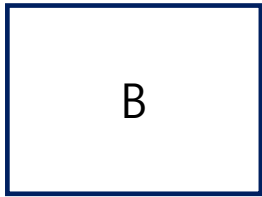
출처: <https://docs-ksc.gitbook.io/nurion-user-guide/undefined/nurion-system-overview-and-configuration>



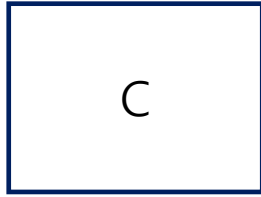
멀티스레드방식 vs. 싱글스레드 방식



대기시간: 15분



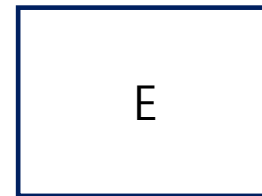
대기시간: 10분



대기시간: 5분



대기시간: 3분



대기시간: 2분

영구의 선택

1. 멀티스레드방식: 맹구, 짱구, 순구, 병구
2. 싱글스레드방식: 호출기

NPM vs NPX

1. npm (Node Package Manager)

- npm은 세계에서 가장 큰 소프트웨어 레지스트리이자, 패키지 관리 도구
- **역할:** 외부 라이브러리(패키지)를 내 프로젝트에 설치(install)하고, 버전을 관리(manage)
- **주요 특징:**
 - ✓ npm install <패키지명>을 통해 라이브러리를 내 컴퓨터에 저장
 - ✓ package.json 파일을 통해 프로젝트가 어떤 라이브러리에 의존하고 있는지 기록
- **한계:** 패키지를 실행하려면 반드시 먼저 설치되어 있어야 하며, 실행 경로를 직접 지정하거나 scripts에 등록해야 하는 번거로움

NPM vs NPX

2. npx (Node Package Execute)

- npx는 npm 5.2.0 버전부터 기본 포함된 패키지 실행기
- **역할:** 패키지를 내 컴퓨터에 영구적으로 설치하지 않고도 즉시 실행할 수 있게 해줌
- **주요 특징:** 일회성 실행: `npx create-react-app my-app`처럼 한 번 쓰고 말 도구를 실행할 때 유용
- **최신 버전 보장:** 로컬에 설치된 버전이 없으면 npm 레지스트리에서 최신 버전을 내려받아 실행하고, 실행 후에는 삭제 (디스크 공간 절약)
- **로컬 바이너리 실행:** `node_modules/.bin` 경로를 일일이 찾지 않아도 로컬에 설치된 패키지를 바로 실행해 줌

NPM vs NPX

3. 핵심 차이점 비교

	NPM	NPX
정의	패키지 관리자(Manager)	패키지 실행기(Runner)
주 목적	패키지 설치 및 업데이트	패키지 일회성 실행
저장 여부	Node_modules 에 영구 저장	실행후 자동 삭제 (임시 설치)
사용 사례	React, Lodash 등 라이브러리 추가	프로젝트 초기 생성 도구(CRA, Vite 등) 실행

결론적으로, npm은 필요한 책을 사서 보고 책꽂이에 보관하는 것이고, npx는 필요한 책을 도서관에서 빌려서 보고 반납하는 것과 유사

2. MCP 실습 환경 설정

□ 주요 설치 프로그램

✓ <https://github.com/rphlee98/KRIBB> 에 접속하셔서 환경구축.md, VS Code 환경설정, 사용자 설정, FastMCP.md 파일 참조

1) Node.js 설치

※ node.js가 이미 설치되어 있는지 또는 잘 설치되었는지 확인:

cmd 창에서 `node -v` 또는 `node --version` 입력

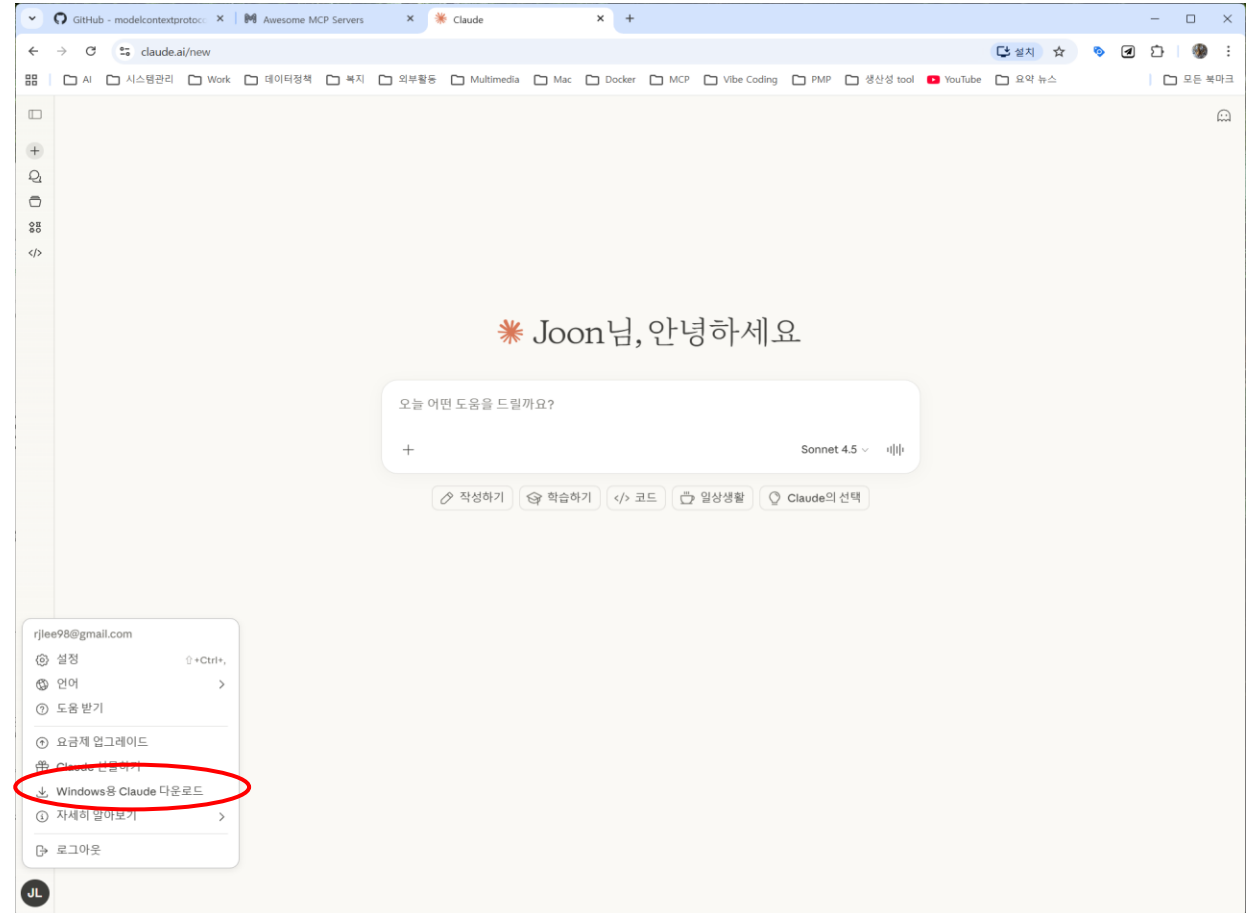


2. MCP 실습 환경 설정

□ 주요 설치 프로그램

2) 클로드 데스크탑 설치

- claude.com/download 사이트에서 Windows 또는 macOS 클릭하여 다운로드 설치
- 또는 claude.com에서 구글 계정으로 연동하고 우측 하단의 아이콘 클릭 후 window용 claude 다운로드 설치



2. MCP 실습 환경 설정

□ 주요 설치 프로그램

3) 파이썬 설치 (보류)

- 다운로드 받은 파이썬을 c:\python314 폴더에 설치
- 윈도우키+R 클릭 > sysdm.cpl 입력 > 고급 탭에서 환경 변수(N) 클릭 > 사용자, 시스템 변수에서 Path 선택 > 편집 클릭 > c:\python314 폴더와 C:\Python314\Scripts\ 폴더를 남겨두고 다른 파이썬 폴더 삭제 > 확인>확인 클릭 후 PC 재부팅

4) MS VS Code 설치

5) 작업 디렉토리 생성 및 이동: c:\project\my_mcp

2. MCP 실습 환경 설정

□ 주요 설치 프로그램

6) 클로드 PC를 열고 왼쪽 상단 메뉴 아이콘 클릭후 파일 > 설정 선택 => 설정에서 개발자 선택 => 구성편집 버튼 클릭 => claud_desktop_config.json 파일 선택 => 오른쪽 마우스 클릭 => 메모장에서 편집 클릭 => “{}” 만 있는 상태 확인(초기 상태)

7) Vs code 실행 => 상단 메뉴에서 파일(file) > 폴더 열기(Open Folder) 클릭 => c:\test 폴더 선택 => Ctrl + ` (백틱기호, 숫자 1 옆의 왼쪽 맨 위 키) 클릭 => vs code 터미널 창 오픈 (또는 상단 메뉴에서 Terminal > New Terminal 클릭)

8) 환경설정 (<https://github.com/rphlee98/KRIBB>)

- VS Code 환경 설정 및 사용자 설정(VS Code 환경설정, 사용자 설정.md 참조)
- uv 설치 ([환경구축.md](#) 파일 참조)

2. MCP 실습 환경 설정

□ 주요 설치 프로그램

9) FastMCP 설치 및 테스트 하기

(https://github.com/rphlee98/KRIBB_FastMCP.md)

- 프로젝트: my_mcp
- 디렉토리: C:\project\my_mcp
- 스크립트: server.py

2. MCP 실습 환경 설정

10) 워밍업

10-1 mcp.tool 실습

(1) python code 작성

```
from mcp.server.fastmcp import FastMCP
#MCP 서버 생성하기
mcp = FastMCP(name="tutorial_1")
@mcp.tool()
def echo(message: str) -> str:
    """
    입력받은 메시지를 그대로 반환하는 도구
    """
    return message + " 라는 메시지가 입력되었습니다. 프로그램
    의 시작은 언제나 Hello World!"
# 서버 실행하기
if __name__ == "__main__":
    mcp.run()
```

(2) 설정파일 작성하기

- claude_desktop_config.json 파일 작성, 위치는
C:\Users\user\AppData\Roaming\Claude 폴더에 위치

```
{
  "mcpServers": {
    "tutorial_1": {
      "command": "python",
      "args": [
        "c:\\MCP_lab\\tutorial_1.py"
      ]
    }
  }
}
```

2. MCP 실습 환경 설정

- AppData 폴더 접근 방법: 윈도우키 누르고 “실행” 입력 => 실행창에 “%appdata%” 입력 > 폴더에서 Claude 선택하면 설정 파일에 접근할 수 있음
- 또는 Claude PC > 파일 > 설정 > 개발자 > 구성편집 클릭 => claude_desktop_config.json 파일에 바로 접근
- 클로드는 작성된 함수를 동적으로 불러오지 않으므로 함수를 제대로 실행하기 위해서는 클로드를 완전히 종료한 후 다시 시작해야 한다.
- 클로드 오른쪽 상단의 X 버튼으로 종료시는 완전히 종료된 것이 아니므로 반드시 파일 > 종료를 선택해서 종료해야 한다.
- 클로드 재실행 후 + 버튼을 눌러 tutorial_1을 클릭 후 제대로 실행되는지 확인 => 해당 서버에 등록된 함수목록 “echo”가 뜨면 정상 실행된 것임. 여기서 함수를 활성화/비활성 시킬 수 있음. 사용하지 않는 함수는 비활성으로 표시해 둘 것
- 연결 실패 메시지가 뜰 경우 MCP설정 열기 > 로그 폴더 열기 > mcp-server-서버이름 파일을 열어 어떤 에러가 발생했는지 확인 가능
- 클로드에서 다음과 같이 입력 후 실행

입력받은 메시지를 그대로 반환해 주는 도구를 활용해서 (또는 echo 함수를 이용해서) hello world를 입력해 줘

- 클로드가 해당 도구를 허용할 것인지를 물어보면 항상 허용 또는 한번만 허용 버튼 클릭

2. MCP 실습 환경 설정

10) 워밍업

10-2 mcp.resource 실습

(1) python code 작성

```
from mcp.server.fastmcp import FastMCP
#MCP 서버 생성하기
mcp = FastMCP(name="tutorial_2")
@mcp.tool()
def add(a: int, b: int) -> int:
    `` 두 숫자를 더하는 함수 ``
    return a + b
@mcp.resource("greeting://hello")
def get_greeting() -> str:
    ``인사말을 제공하는 함수``
    return f"Hello, world!"
# 서버 실행하기
if __name__ == "__main__":
    mcp.run()
```

(2) 설정파일 작성하기

- claude_desktop_config.json 파일 작성, 위치는 C:\Users\user\AppData\Roaming\Claude 폴더에 위치

```
{
  "mcpServers": {
    "tutorial_1": {
      "command": "python",
      "args": [
        "c:\\W\\MCP_lab\\W\\tutorial_1.py"
      ]
    },
    "tutorial_2": {
      "command": "python",
      "args": [
        "c:\\W\\MCP_lab\\W\\tutorial_2.py"
      ]
    }
  }
}
```

2. MCP 실습 환경 설정

- 파일 > 종료를 통해 Claude 재실행하고 + 아이콘 클릭하여 서버가 제대로 로드 되었는지 확인
- Claude 프롬프트
 - add 라는 함수를 이용해서 a=10, b=30을 더해줘
- Resource로 구현된 함수를 직접 호출하면 claude가 해당 함수를 실행할 수 없다고 하면서 대신 실행 가능한 함수 목록을 제시
- “get_greeting에 name은 영구를 넣어줘”라고 테스트해 볼 것
- 이는 Resource 가 함수 호출 방식이 아닌 주소 기반 접근 방식을 사용하는 방식에 기인함
- 따라서 resource 에 접근하기 위해서는 대화창 하단의 + 버튼 클릭 > 표시된 메뉴에서 tutorial_2에서 추가 선택 > 등록된 resource 목록에서 get_greeting 선택
- 등록한 resource가 파일 형태로 대화창에 추가되면 첨부된 파일 오픈 > 파일은 return 값 “hello, world!”를 지님

2. MCP 실습 환경 설정

10) 워밍업

10-3 mcp.prompt 실습

(1) python code 작성

```
from mcp.server.fastmcp import FastMCP
#MCP 서버 생성하기
mcp = FastMCP(name="tutorial_3")
# Prompt 확장 예제
@mcp.prompt()
def prompt_extension(contents: str) -> str:
    `` 프롬프트에서 사실과 의견을 구분합니다. ``
    return f``{contents}
    이 프롬프트에 대해 아래와 같은 템플릿 서식에 맞도록 답변해
    줘.
    * 사실:
    * 의견:
    `` 코드 샘플 제공 ``
# 서버 실행하기
if __name__ == "__main__":
    mcp.run()
```

(2) 설정파일 작성하기

- claude_desktop_config.json: C:\Users\user\AppData\Roaming\Claude 폴더에 위치

```
{
  "mcpServers": {
    "tutorial_1": {
      "command": "python",
      "args": [
        "c:\\MCP_lab\\tutorial_1.py"
      ]
    },
    "tutorial_2": {
      "command": "python",
      "args": [
        "c:\\MCP_lab\\tutorial_2.py"
      ]
    },
    "tutorial_3": {
      "command": "python",
      "args": [
        "c:\\MCP_lab\\tutorial_3.py"
      ]
    }
  }
}
```

2. MCP 실습 환경 설정

- 파일 > 종료를 통해 Claude 재실행하고 + 아이콘 클릭하여 서버가 제대로 로드되었는지 확인
- 대화창 하단의 + 버튼 클릭, tutorial_3에서 추가 선택 > prompt_extension 클릭
- 프롬프트 입력 창이 나타나면 프롬프트 입력후 프롬프트 추가 버튼 클릭
- Claude 프롬프트
2026년 인공지능 전망에 대해 알려줘
- 첨부된 파일 클릭하여 양식 확인

[질문] 설정 파일에 JSON 파일을 쓰는데 JSON 파일은 무엇인가?

○ JSON(JavaScript Object Notation)이란

"JSON(JavaScript Object Notation)"은 데이터를 저장하거나 전송할 때 사용하는 가장 대중적인 **텍스트 기반의 데이터 포맷**입니다.

이름에 'JavaScript'가 들어가지만, 사실상 거의 모든 프로그래밍 언어에서 지원하기 때문에 현대 웹과 앱 개발에서 데이터를 주고받는 표준으로 자리 잡았습니다.

1. JSON의 핵심 특징

- **가독성:** 사람이 읽고 쓰기 쉽고, 기계가 분석하고 생성하기에도 편리합니다.
- **경량성:** XML과 같은 다른 형식에 비해 구조가 단순하여 데이터 크기가 작습니다.
- **독립성:** 특정 프로그래밍 언어에 종속되지 않는 독립적인 데이터 포맷입니다.

2. JSON의 기본 구조

JSON은 크게 두 가지 구조를 조합하여 만듭니다.

- 이름(Key)과 값(Value)의 쌍: { "key": "value" } 형태 (객체)
- 정렬된 목록: ["value1", "value2"] 형태 (배열)

3. 사용 가능한 데이터 타입

JSON에서는 다음과 같은 데이터 타입을 사용할 수 있습니다.

타입	설명	예시
String	큰따옴표(")로 감싼 문자열	"name": "Gemini"
Number	정수 또는 실수	"age": 25, "price": 9.99
Object	중괄호({})로 감싼 키-값 쌍의 집합	{"city": "Seoul"}
Array	대괄호([])로 감싼 값들의 목록	["apple", "banana"]
Boolean	참 또는 거짓	"is_active": true
null	빈 값을 의미	"middle_name": null

4. 실제 예시

사용자의 정보를 JSON으로 표현하면 다음과 같습니다.

```
{
  "user_id": 101,
  "username": "Joon_Lee",
  "is_premium": true,
  "skills": ["Python", "JavaScript", "SQL", "Node.js"],
  "address": {
    "city": "Daejeon",
    "zipcode": "34141"
  }
}
```

5. 주요 활용 사례

- **웹 API:** 서버와 브라우저 간에 데이터를 주고받을 때 주로 사용합니다. (예: 날씨 정보 불러오기)
- **설정 파일:** VS Code 설정이나 프로젝트의 package.json처럼 프로그램의 환경 설정을 저장할 때 사용합니다.
- **NoSQL 데이터베이스:** MongoDB와 같은 DB는 데이터를 JSON과 유사한 형태로 저장합니다.

주의사항: JSON에서는 반드시 **큰따옴표(")**만 사용해야 하며, 마지막 데이터 뒤에 **쉼표(,)**를 붙이지 않도록 주의해야 합니다.

4. 개발 환경 설정

4.1 VSCode + GitHub Copilot에서 MCP 서버 설정

○ VSCode에서 MCP 서버 설정 방법

방법	파일 위치	적용 범위
사용자 설정	VS Code settings.json	모든 프로젝트에 적용
프로젝트 설정	.vscode/mcp.json	해당 프로젝트에만 적용

- 팀과 함께 작업하는 경우 .vscode/mcp.json을 Git에 포함시키면 팀원 모두가 동일한 MCP 서버 환경 사용이 가능

○ settings.json에 MCP 서버 추가하기

- VS Code의 사용자 설정(settings.json)에 직접 MCP 서버를 추가하는 방법

1. Ctrl + Shift + P(macOS: Cmd + Shift + P)로 명령 팔레트를 엽니다.
2. Preferences: Open User Settings (JSON)을 선택합니다.
3. 다음과 같이 mcp 섹션을 추가합니다

```
{
  "mcp": {
    "servers": {
      "github": {
        "command": "npx",
        "args": [
          "-y",
          "@modelcontextprotocol/server-github"
        ],
        "env": {
          "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_xxxxxxxxxxxx"
        }
      }
    }
  }
}
```

4. 개발 환경 설정

4.1 VSCode + GitHub Copilot에서 MCP 서버 설정

- settings.json에 토큰을 직접 넣는 것은 보안상 권장되지 않음. 대신 "env" 항목에서 시스템 환경 변수를 참조하는 방식 사용. VS Code는 \${env:변수명} 구문 지원

```
"env": {  
  "GITHUB_PERSONAL_ACCESS_TOKEN": "${env:GITHUB_TOKEN}"  
}
```

- .vscode/mcp.json 파일 생성을 통해 해당 프로젝트에서만 사용할 MCP 서버를 정의

```
{  
  "servers": {  
    "filesystem": {  
      "command": "npx",  
      "args": [  
        "-y",  
        "@modelcontextprotocol/server-filesystem",  
        "/Users/username/projects/my-project"  
      ]  
    },  
    "github": {  
      "command": "npx",  
      "args": [  
        "-y",  
        "@modelcontextprotocol/server-github"  
      ],  
      "env": {  
        "GITHUB_PERSONAL_ACCESS_TOKEN": "${env:GITHUB_TOKEN}"  
      }  
    }  
  }  
}
```

👉 이 파일을 .gitignore에 포함시키지 않으면 팀원들과 MCP 서버 설정을 공유할 수 있음. 단, 토큰 같은 민감 정보는 반드시 환경변수로 분리해야 함.

4. 개발 환경 설정

4.1 VSCode + GitHub Copilot에서 MCP 서버 설정

○ GitHub Copilot Chat에서 MCP 서버 활용하기

- MCP 서버가 설정되면 GitHub Copilot Chat의 **Agent Mode**에서 자동 인식

1. Copilot Chat 패널을 엽니다(Ctrl + Shift + I).
2. 채팅 모드를 **Agent**로 전환합니다(채팅 입력창 상단 드롭다운).
3. 연결된 MCP 서버의 Tool 목록이 도구 아이콘(🔧)을 클릭하면 표시됩니다.
4. 자연어로 질문하면 AI가 필요한 Tool을 자동으로 선택하여 호출합니다.

사용자: "이 리포지토리에서 최근 열린 이슈들을 보여줘"

AI → MCP Tool 호출: `github.list_issues({owner: "my-org", repo: "my-repo", state: "open"})`

→ 결과를 자연어로 정리하여 응답

- VS Code에서 MCP 서버가 제대로 인식되지 않으면, 명령 팔레트에서 MCP: List Servers를 실행하여 등록된 서버 목록과 상태를 확인할 수 있음. 서버 상태가 "not started"로 표시되면 해당 서버를 클릭하여 수동으로 시작할 수 있음.

4. 개발 환경 설정

4.2 Claude Desktop에서 MCP 서버 설정하기

- Claude Desktop은 Anthropic이 제공하는 데스크톱 애플리케이션으로, MCP를 제일 처음으로 지원한 Host임. 설정이 비교적 간단하고 안정적이어서 MCP 입문용으로 적합
- Claude_Desktop_config.json 파일 위치
 - Claude Desktop의 MCP 서버 설정은 JSON 파일 하나로 관리됨. 운영체제별 파일 위치는 다음과

같음

운영체제	설정 파일 경로
Windows	%APPDATA%\Claude\claude_desktop_config.json
macOS	~/Library/Application Support/Claude/claude_desktop_config.json
Linux	~/.config/Claude/claude_desktop_config.json

- Windows에서는 파일 탐색기 주소창에 %APPDATA%\Claude를 입력하면 바로 해당 폴더로 이동할 수 있음.
- 해당 경로에 파일이 없다면 직접 생성하면 됨. Claude Desktop을 처음 실행한 경우 설정 파일이 아직 만들어지지 않았을 수 있음.

4. 개발 환경 설정

4.2 Claude Desktop에서 MCP 서버 설정하기

○ 서버 추가 설정 예제

- claude_desktop_config.json 파일을 열고 다음과 같이 MCP 서버를 추가

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-file-system",
        "C:\\Users\\username\\Documents"
      ]
    },
    "github": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-github"
      ],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_xxxxxxxxxxxxx"
      }
    },
    "sqlite": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-sqlite",
        "C:\\Users\\username\\data\\sales.db"
      ]
    }
  }
}
```

- 설정 파일을 수정한 후에는 반드시 Claude Desktop을 재시작해야 변경사항이 적용됨.

4. 개발 환경 설정

4.2 Claude Desktop에서 MCP 서버 설정하기

○ 서버 연결 상태 확인 방법

- Claude Desktop에서 MCP 서버가 정상적으로 연결되었는지 확인하는 방법은
 1. Claude Desktop을 재시작합니다.
 2. 새 대화를 시작합니다.
 3. 채팅 입력창 하단에 **MCP 도구 아이콘**(망치 모양 )이 표시되는지 확인합니다.
 4. 아이콘을 클릭하면 연결된 서버와 사용 가능한 Tool 목록이 나타납니다.
- 정상적으로 연결되면 다음과 같이 Tool 목록이 표시됨.

```
📁 filesystem
- read_file
- write_file
- list_directory
- search_files
...
🌸 github
- list_repositories
- search_issues
- get_pull_request
...
```

4. 개발 환경 설정

4.2 Claude Desktop에서 MCP 서버 설정하기

○ 자주 발생하는 오류와 해결 방법

증상	원인	해결 방법
서버 아이콘이 표시되지 않음	JSON 문법 오류	JSON 유효성 검사기로 설정 파일 확인
"spawn npx ENOENT" 오류	Node.js가 설치되지 않음	Node.js 18 이상 설치
서버가 시작 후 바로 종료	토큰이 잘못됨 또는 패키지 설치 실패	터미널에서 동일 명령 직접 실행하여 오류 확인
"TIMEOUT" 오류	네트워크 문제로 npx 패키지 다운로드 실패	네트워크 확인 후 재시도
Windows에서 경로 오류	백슬래시 이스케이프 누락	경로에 \\\\ 사용 또는 / 사용

- Claude Desktop의 로그 파일에서 상세한 오류 정보를 확인할 수 있음.
 - **Windows:** %APPDATA%\Claude\logs\mcp*.log
 - **macOS:** ~/Library/Logs/Claude/mcp*.log
- 로그 파일을 열면 서버 시작 과정에서 발생한 오류 메시지를 직접 확인할 수 있음.

4. 개발 환경 설정

○ MCP 지원 현황

기능	VS Code + Copilot	Claude Desktop	Cursor	Windsurf
MCP 지원	O (Agent Mode)	O (네이티브)	O (네이티브)	O (Cascade)
설정 파일	settings.json 또는 .vscode/mcp.json	claude_desktop_config. json	~/.cursor/mcp.json	mcp_config.json
프로젝트별 설정	.vscode/mcp.json	미지원	.cursor/mcp.json	미지원
환경변수 참조	\${env:VAR} 지원	직접 입력	직접 입력	직접 입력
서버 상태 확인	MCP: List Servers	도구 아이콘	Settings → MCP	Cascade UI
Tool 승인 방식	호출 시 확인 팝업	호출 시 확인 팝업	호출 시 확인 팝업	자동 또는 확인
Sampling 지원	제한적	O	제한적	제한적

4. 개발 환경 설정

4.3 MCP Inspector로 서버 상태 점검하기

○ MCP Inspector란

- MCP Inspector는 MCP 서버를 대화형으로 테스트하고 디버깅할 수 있는 웹 기반 도구로 Anthropic 이 공식 제공하며, MCP 서버 개발자와 사용자 모두에게 필수적인 도구
- MCP 서버를 설정했는데 제대로 동작하지 않을 때, 또는 서버가 어떤 Tool과 Resource를 제공하는 지 확인하고 싶을 때 사용하는 도구
- 핵심 기능:
 - 서버가 제공하는 **Tool 목록** 확인
 - 서버가 제공하는 **Resource 목록** 확인
 - **Tool**을 직접 호출하여 결과 확인
 - 서버 로그를 실시간으로 모니터링
 - JSON-RPC 메시지를 직접 확인

4. 개발 환경 설정

4.3 MCP Inspector로 서버 상태 점검하기

○ MCP Inspector 실행하기

- 별도의 설치 없이 npx로 즉시 실행 가능

```
npx @modelcontextprotocol/inspector
```

- 실행하면 웹 브라우저에서 Inspector UI가 열림. 기본적으로 <http://localhost:6274>에서 실행
- 특정 MCP 서버를 바로 연결하려면 다음과 같이 서버 명령을 인자로 전달

```
npx @modelcontextprotocol/inspector npx @modelcontextprotocol/server-github
```

4. 개발 환경 설정

4.3 MCP Inspector로 서버 상태 점검하기

- 환경변수가 필요한 경우 -e 플래그를 사용

```
npx @modelcontextprotocol/inspector ₩  
-e GITHUB_PERSONAL_ACCESS_TOKEN=ghp_xxxxxxxxxxxx ₩  
npx @modelcontextprotocol/server-github
```

- Inspector UI에서 Tools 탭을 클릭하면 서버가 제공하는 모든 Tool 목록 확인
이 가능
- resources/list 탭에서 Resource 목록 확인이 가능

```
Resources:  
📄 schema://sales - Sales table schema  
📄 schema://customers - Customers table schema  
📄 schema://products - Products table schema
```

4. 개발 환경 설정

4.5 MCP Inspector로 서버 상태 점검하기

- Tools/call로 Tool을 직접 호출할 수 있으며, 특정 tool을 선택하고 파라미터를 입력한 후 Run 버튼을 누르면 실제 결과 확인이 가능
- 예를 들어 search_issues Tool을 테스트한다면 다음과 같이 진행합니다.
 1. Tools 탭에서 search_issues를 클릭합니다.
 2. 파라미터 입력란에 값을 입력합니다.
 3. **Run** 버튼을 클릭합니다.
 4. 결과가 JSON 형태로 표시됩니다.

```
{  
  "query": "repo:anthropics/anthropic-sdk-python is:open label:bug"  
}
```

```
{  
  "content": [  
    {  
      "type": "text",  
      "text": "Found 5 issues:\n1. #234 - Connection timeout in streaming mode\n2. #221 - Token counting mismatch\n..."  
    }  
  ]  
}
```

MCP 이해와 활용(실습 I)

오후수업 (12:30~16:50)

0-2 강의 진행

❖ 오전

- MCP의 이론 및 관련 기술에 대한 소개
- 실습 환경 구축과 에러시 대응 방안, 간단한 실습

❖ 오후

- 외부에서 MCP 서버 가져다 쓰는 방식 실습
- MCP 코드 작성 및 직접 설정 방식 위주의 예제 실습
- 반복업무 자동화 및 직무별 활용사례

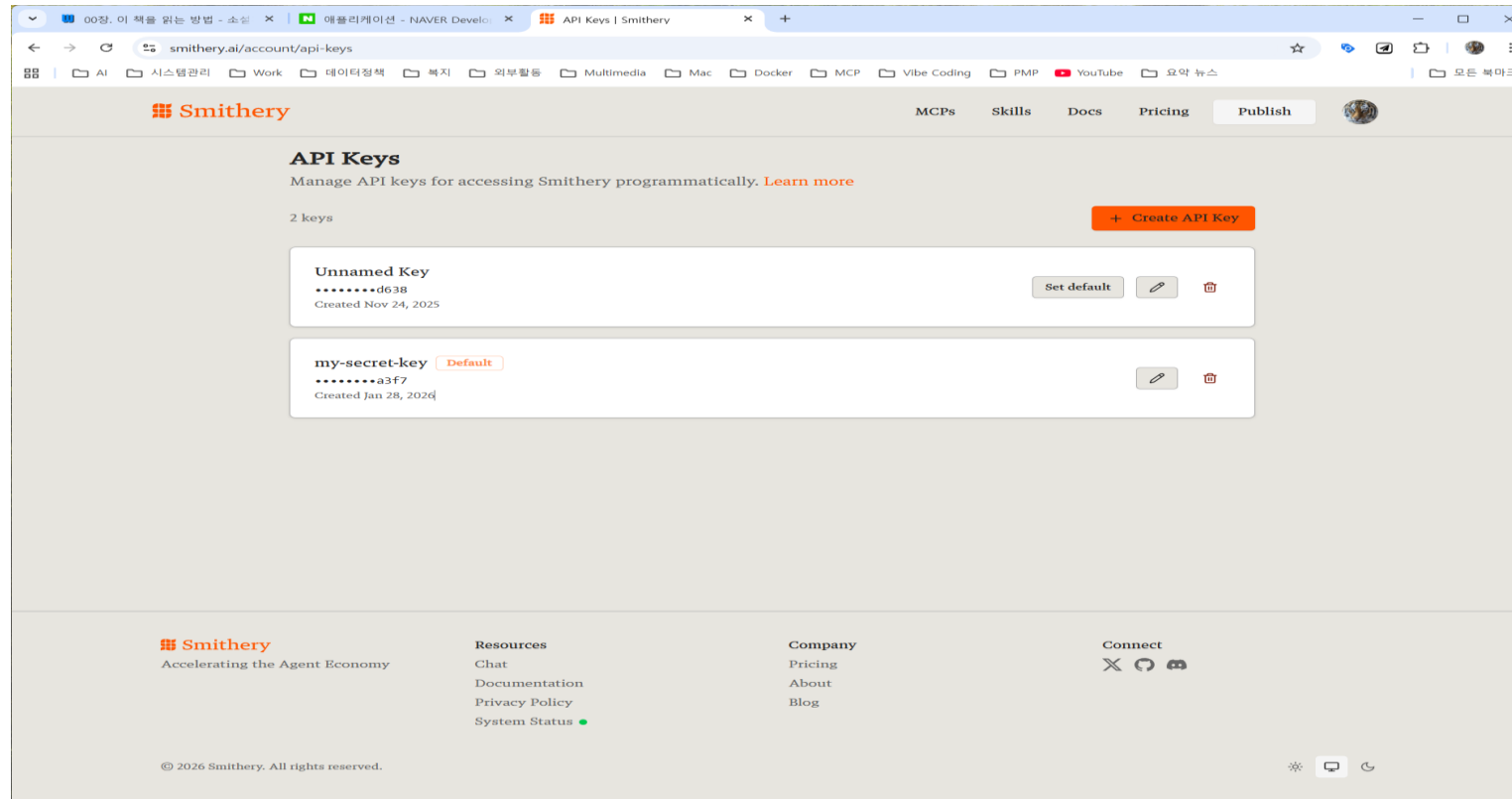
5. MCP 실습(1)

1) 바탕화면 정리하기

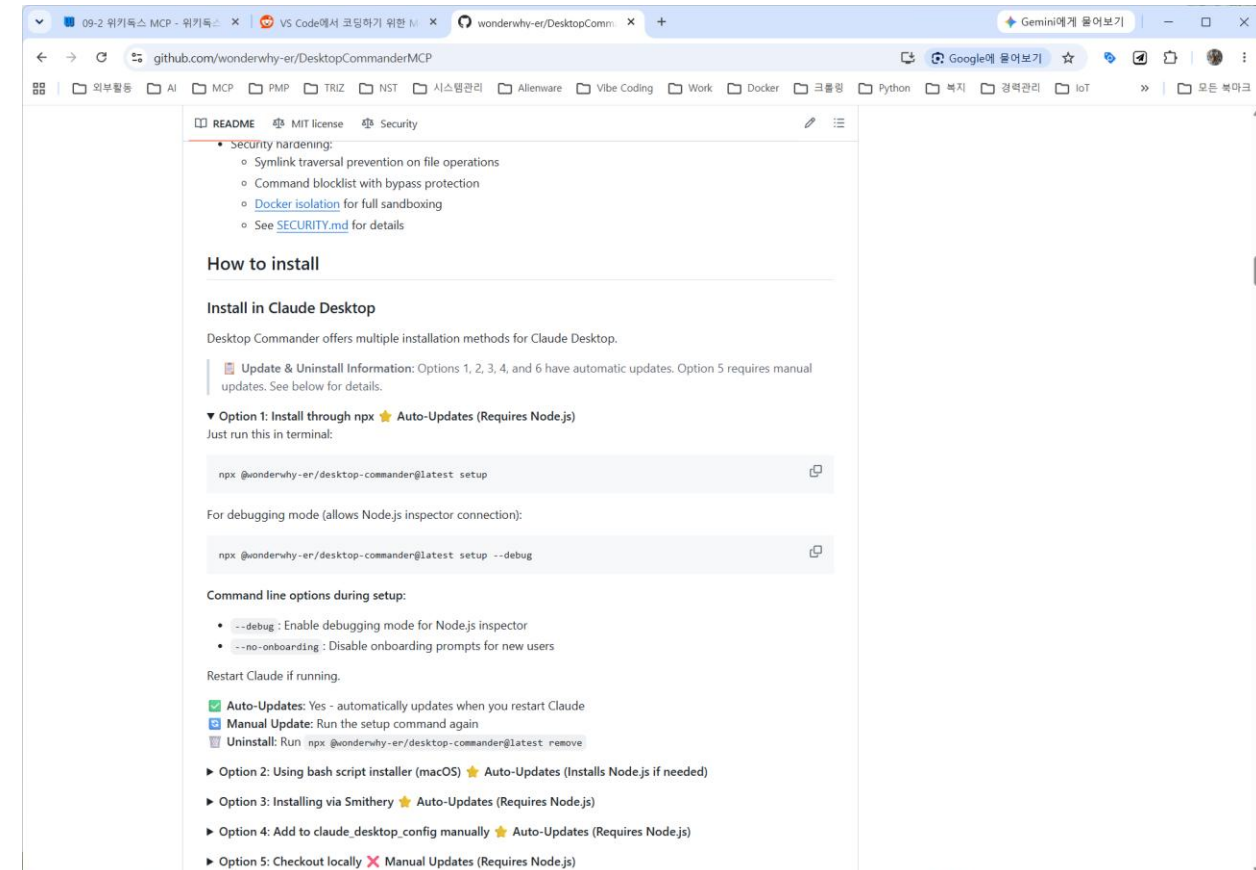
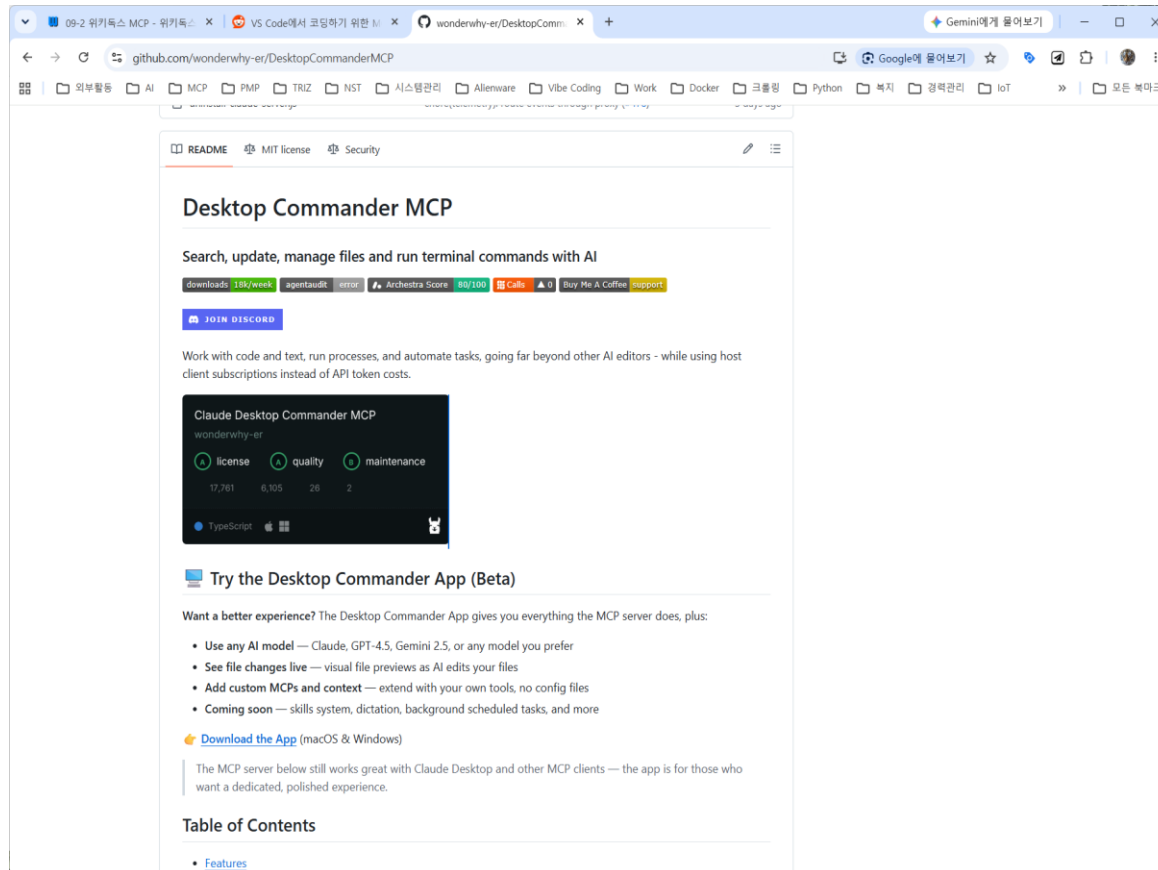
- 기존 바탕화면 파일을 복사해서 "original" 폴더로 이동
- 실습용 바탕화면 예제 설치 (<https://github.com/rphlee98/MCP-lab>)
- [바탕화면 오른쪽 클릭 ⇒ 경로로 복사] 선택
- 클로드 데스크 탑 오픈
- "C:\Users\DEV\Desktop" 위치에 있는 파일을 알아서 잘 분류하여 폴더에 정리 해줘"
- Desktop Commander MCP 로 바탕화면을 정리하려고 권한을 요청 => [이 대화에 허용]을 눌러 작업 수행
- 잠깐!! 이후 2)~3) 실습 진행 후 다시 돌아옵니다. 먼저 폴더 목록 만드는 실습부터 진행하고 돌아와서 가져다 쓰는 실습을 하겠습니다. 가져다 쓰는 실습은 다음 [참고사항] 참조하세요

[참고사항] 스미더리 개인 API 설정하기

- smithery.ai 접속
- 구글 계정 등을 통해 회원가입, 로그인
- 상단의 [프로필 사진] 클릭후 API Keys 선택
- + Create API Key 눌러서 키 활성화하고 키 값을 복사하여 안전하게 저장



- 구글에서 'Desktop Commander MCP' 검색후 사이트 이동(<https://github.com/wonderwhy-er/DesktopCommanderMCP>)
- 하단 How to install – Option 1: Install through npx 선택
- 터미널에 npx @wonderwhy ... 내용을 복사하여 붙여넣은 다음 실행
- 클로드 데스크탑을 종료후 다시 실행한 뒤 'Desktop Commander MCP'가 서버 목록에 있는지 확인하면 이후 데스크탑 커맨드를 사용 가능



2) PYTHON 코드로 폴더 생성, 삭제, 목록 조회하기

- python 코드 작성하기 (실습자료.md 파일 참조)
- vs code 터미널에서 python test.py 실행
- 생성된 폴더 확인, 이후 폴더 생성을 주석 처리하고 폴더 삭제 주석 해제 후 재 실행하여 결과 확인 해 볼 것

3) 파일 입출력 실습

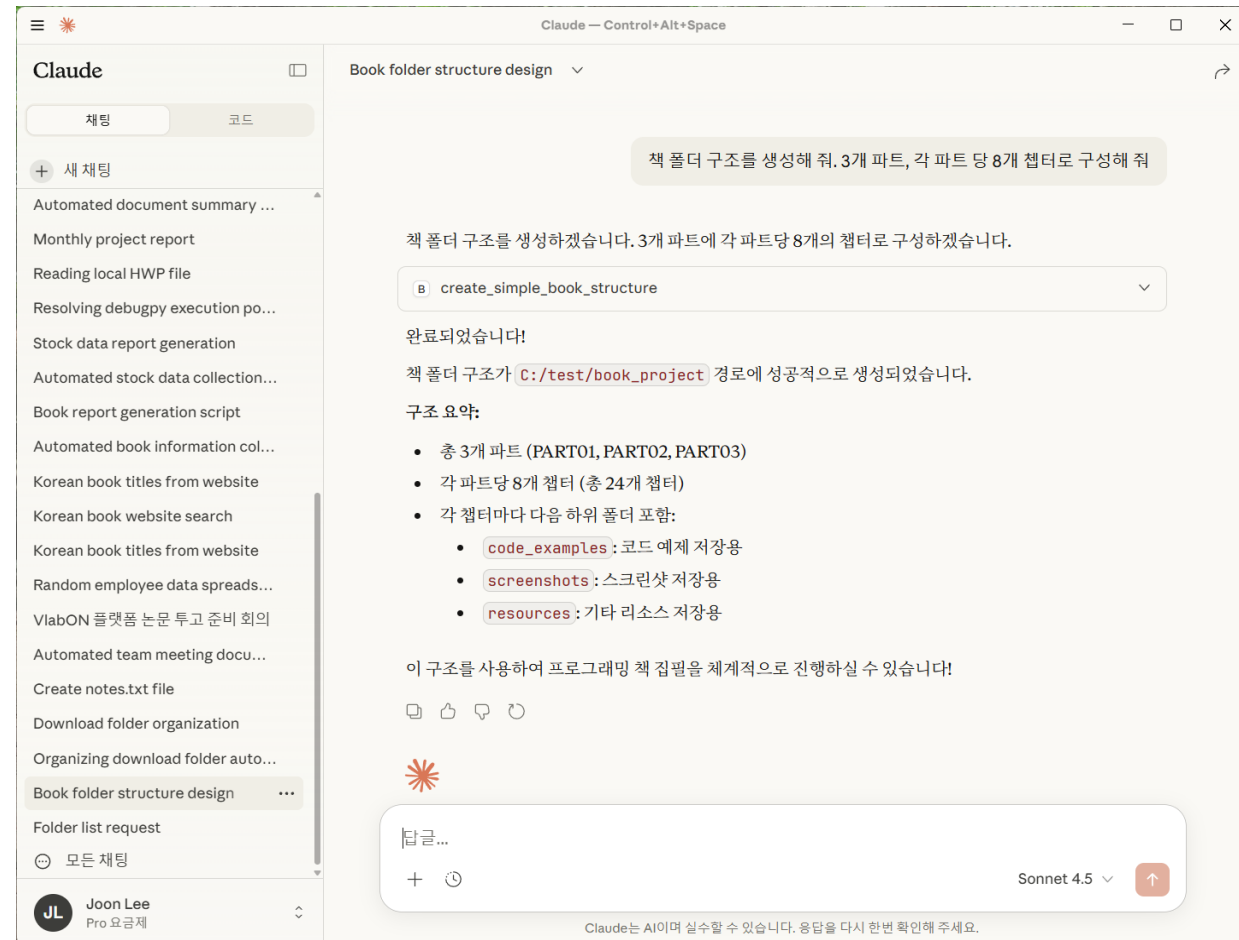
- 프롬프트 작성하기 (실습자료.md 파일 참조)
- 작성된 코드를 server.py으로 저장
- 생성된 폴더 확인, 이후 폴더 생성을 주석 처리하고 폴더 삭제 주석 해제 후 재 실행하여 결과 확인해 볼 것
- MCP 설정파일(claude_desktop_config.json)을 열고 "{}"만 남겨놓고 모두 삭제
- 터미널을 열고 다음 명령 입력
mcp install server.py
- 의미는 MCP로 server.py 파일을 등록하려고 하니 설정파일에 server.py 파일을 등록해 줘
- 에러시 **pip install uv** 명령어로 uv 설치 후 재 실행
- 설정파일을 다시 열어서 우측과 같은 내용이 등록되어 있으면 성공

```
{
  "mcpServers": {
    "mcp_project": {
      "command": "uv",
      "args": [
        "run",
        "--with",
        "mcp[cli]",
        "mcp",
        "run",
        "C:\\\\test\\\\server.py"
      ]
    }
  }
}
```

- 설정 완료 후 파일 > 종료 선택하고 클로드 재 실행
- 다음 프롬프트 차례로 입력 후 확인
 - 폴더 목록을 읽어 줘
 - mcp_test 라는 폴더를 생성해 줘
 - 폴더 목록을 읽어줘
 - mcp_test 폴더를 삭제해 줘
 - 폴더 목록을 다시 읽어 줘

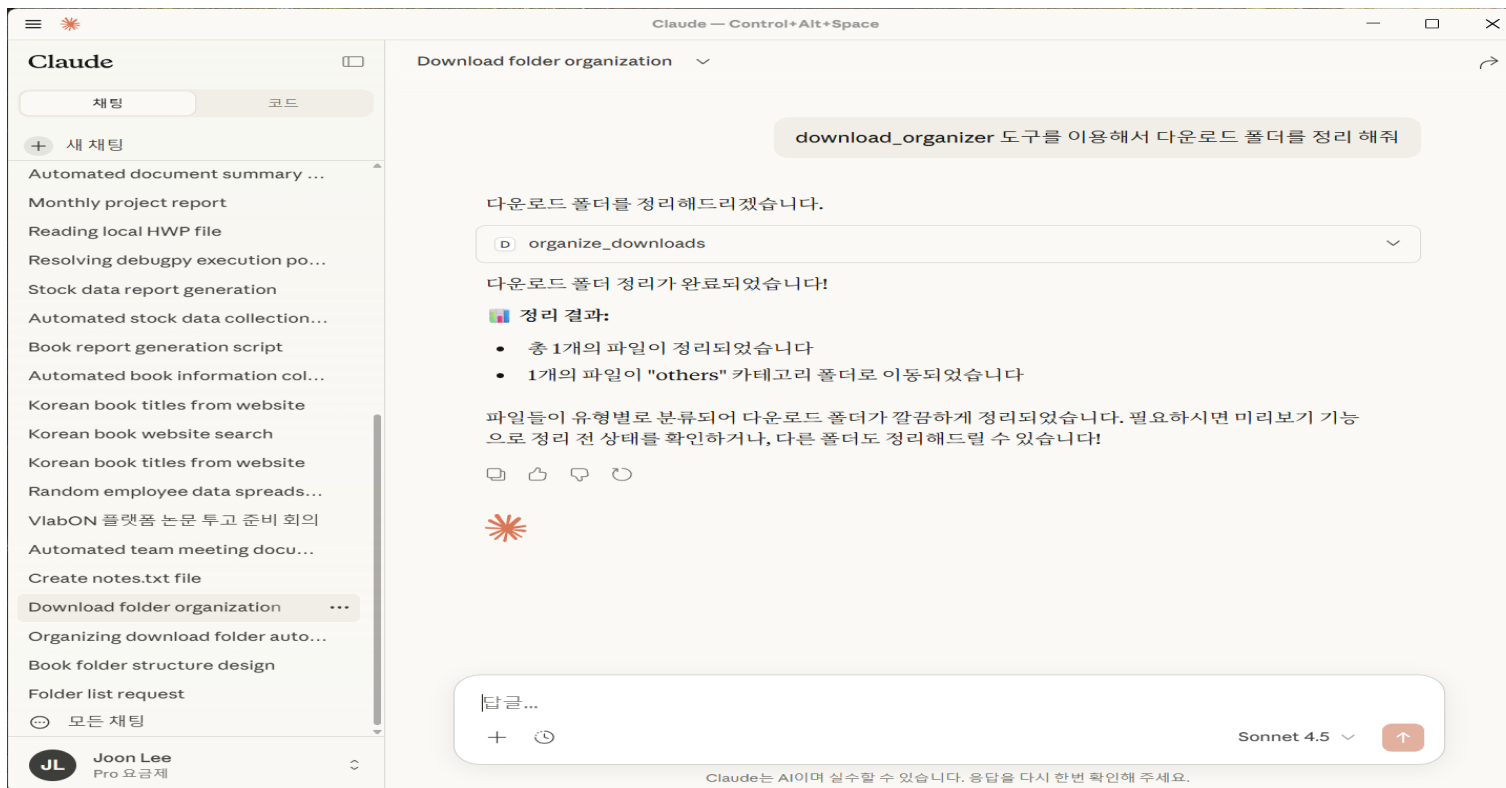
[응용예제 1] 책 예제 폴더 만들기

- Claude를 열고 다음 프롬프트 작성
- 참조할 소스 코드는 키보드 숫자 1 옆의 백틱키(`) 3개를 입력하고 shift+enter 키를 누르면 그 아래 텍스트가 코드 블록으로 변경되는데 거기에 기존에 작성한 코드를 추가하면 된다 (md 문법)
- 실습자료.md 파일에서 복사해서 쓰되, 백틱 3개는 (```)는 ``` 로 수정해서 사용할 것. 왜 그럴까?
- 클로드가 생성한 코드를 등록하고 재 실행후 다음 프롬프트 입력하여 결과 확인해 볼것
- 잠깐!! 코드 실행후 왜 설정파일 등록을 안해줄까?



[응용예제 2] 다운로드 폴더 정리하기

- Claude를 열고 프롬프트 작성 (실습자료.md 파일 참조)
- 참조할 소스 코드는 키보드 숫자 1 옆의 백틱키(`) 3개를 입력하고 shift+enter 키를 누르면 그 아래 텍스트가 코드 블록으로 변경되는데 거기에 기존에 작성한 코드를 추가하면 된다.
- 클로드가 생성해 준 다운로드 폴더 정리 MCP서버 코드 (실습자료.md 파일 참조)



MCP Repository

- <https://github.com/modelcontextprotocol/servers>
- <https://www.pulsemcp.com/servers>
- <https://smithery.ai/>
- <https://mcp.so/>
- <https://mcpservers.org/>
- <https://playmcp.kakao.com/>

6. MCP 실습(2): 가져다 쓰는 MCP서버

1) Smithery: MCP 서버의 중앙 레지스트리

○ Smithery란

- Smithery는 스스로를 "MCP 서버를 위한 Hugging Face"로 정의
- 2025년 8월 현재 5,930개가 넘는 MCP 서버 보유
- 특징: 클라이언트와 무관(client-agnostic)한 설계 방식을 채택함으로써, Claude Desktop, Cursor, VS Code, LM Studio 등 어떠한 MCP 클라이언트에서든 Smithery의 서버 활용이 가능

○ Smithery 배포 모델: 로컬 설치 및 호스팅 방식

(1) 로컬 설치(Local Installation)

- 로컬 설치: MCP 서버를 사용자의 컴퓨터에서 직접 실행하는 방식
- Smithery CLI를 통해 서버를 다운로드하고 로컬 환경에서 구동
- 장점: 완전한 사용자 제어, API키와 같은 민감한 정보가 사용자의 머신을 벗어나지 않으므로 최고 수준의 보안 제공
- Smithery는 사용자가 서버를 설치했다는 사실만 통계 목적으로 기록할 뿐, 실제 설정 데이터나 토큰을 수집하지 않음

6. MCP 실습(2): 가져다 쓰는 MCP서버

(1) 로컬 설치(Local Installation)

```
# 기본 설치 명령
npx @smithery/cli install mcp-obsidian --client claude

# 설정과 함께 설치하여 프롬프트 생략
npx @smithery/cli install mcp-obsidian --client claude --config
'{"vaultPath":"path/to/vault"}'
```

(2) 호스티드 서버(Hosted/Remote)

- 호스티드 방식은 Smithery의 인프라에서 MCP 서버가 실행되고, 사용자는 웹을 통해 즉시 접근할 수 있는 모델로 서버관리나 의존성 설치 없이 바로 사용할 수 있다는 장점이 있으나 보안과 프라이버시 관점에서는 사용자의 주의가 필요
- 호스티드 서버를 사용할 때는 API 키가 Smithery의 백엔드로 전달되며, Smithery는 이러한 설정 데이터를 장기간 저장하지 않고 일시적으로만 처리한다고 명시함. 그러나 서버가 작동하는 동안은 호출 메타데이터를 확인할 수 있어서 완전한 프라이버시가 필요한 경우는 로컬 설치를 권장

6. MCP 실습(2): 가져다 쓰는 MCP서버

○ Smithery CLI

- 핵심 관리 명령어들

```
# 설치된 서버 목록 확인
npx @smithery/cli list servers --client claude

# 서버 상세 정보 조회
npx @smithery/cli inspect mcp-obsidian

# 서버 제거
npx @smithery/cli uninstall mcp-obsidian --client claude

# 지원하는 클라이언트 목록
npx @smithery/cli list clients

# 디버깅용 상세 로그
npx @smithery/cli install mcp-obsidian --client claude --verbose
```

※ 특히 inspect 명령어는 서버를 실제로 설치하기 전에 기능을 테스트해 볼 수 있는 대화형 환경을 제공하므로, 이를 통해 서버의 품질을 사전에 검증하고 자신의 요구사항에 맞는지 확인할 수 있는 중요한 기능임.

6. MCP 실습(2): 가져다 쓰는 MCP서버

○ Smithery CLI

- 개발자를 위한 도구들

```
# 개발 서버 시작 (핫 리로드 지원)
```

```
npx @smithery/cli dev
```

```
# 프로덕션 빌드
```

```
npx @smithery/cli build
```

```
# 플레이그라운드 열기
```

```
npx @smithery/cli playground
```

```
# 서버 배포
```

```
smithery deploy .
```

6. MCP 실습(2): 가져다 쓰는 MCP서버

○ Smithery CLI

- 보안을 고려한 환경변수 활용: API키나 민감한 정보는 절대 명령어에 입력하지 말고 환경변수를 사용할 것을 권장

잘못된 방법

```
npx @smithery/cli install github-server --config '{"token":"ghp_xxxxxxxxxxxxx"}'
```

올바른 방법

```
export GITHUB_TOKEN="ghp_xxxxxxxxxxxxx"
```

```
npx @smithery/cli install github-server --config '{"token":"$GITHUB_TOKEN"}'
```

6. MCP 실습(2): 가져다 쓰는 MCP서버

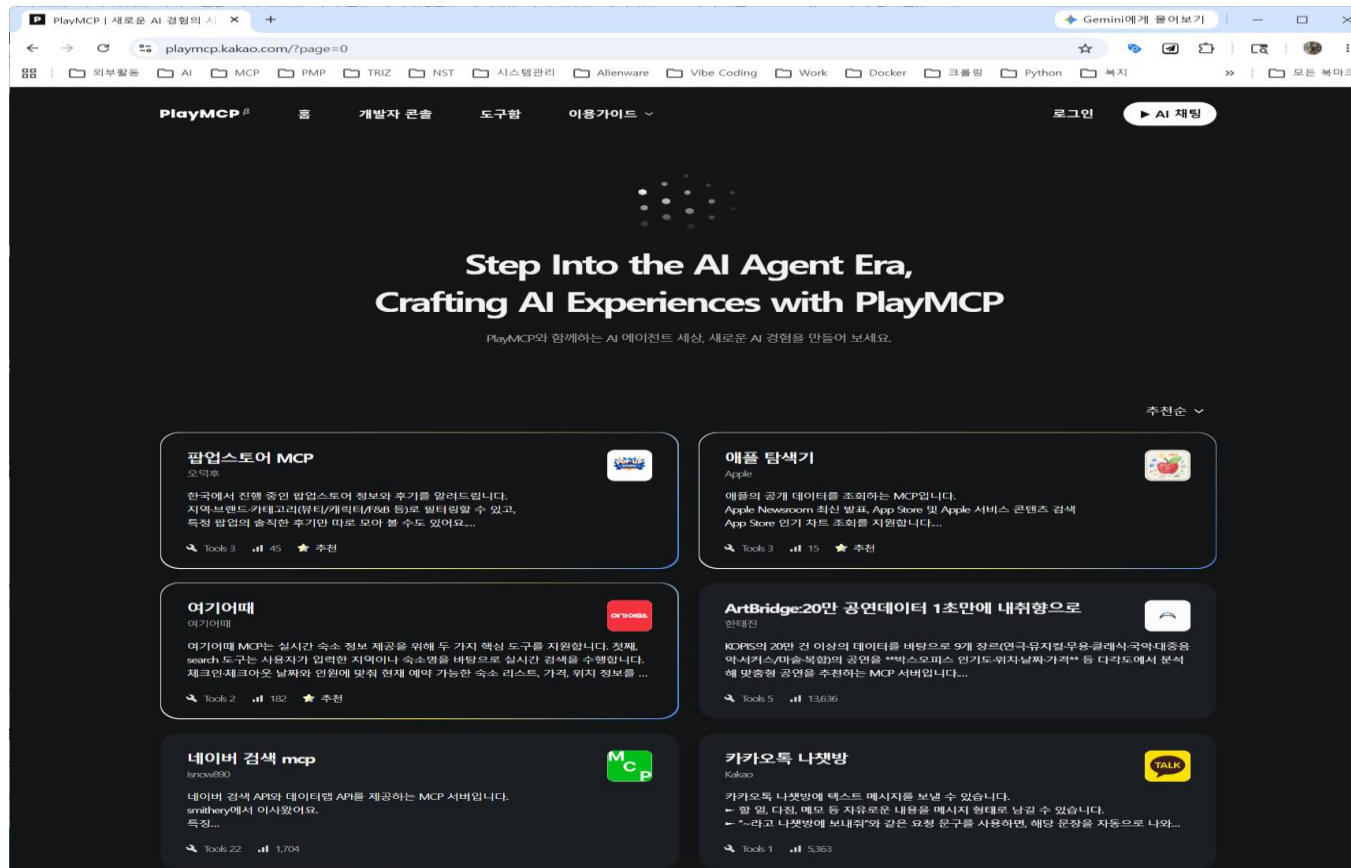
○ Smithery의 생태계적 가치

- Smithery의 진정한 가치는 단순히 MCP 서버를 모아놓은 저장소로서의 기능에서 더 나아가 “**생태계 전체의 성장을 가속화하는 인프라**”라는 점
- **네트워크 효과:** 한 사람이 유용한 MCP 서버를 만들면, 전 세계 수천 명의 개발자들이 즉시 활용할 수 있습니다. 이러한 네트워크 효과는 개발자들에게 기여할 동기를 제공하고, 결과적으로 전체 생태계의 질을 향상시킵니다.
- **표준화의 힘:** Smithery는 MCP 서버의 패키징, 배포, 버전 관리에 대한 사실상의 표준을 제시합니다. 이는 개발자들이 일관된 방식으로 서버를 만들고 배포할 수 있게 하여, 사용자 경험을 크게 개선합니다.
- **품질관리:** 커뮤니티 기반의 리뷰 시스템과 사용량 통계를 통해 자연스럽게 품질이 높은 서버들이 상위에 노출됩니다. 월간 사용량이라는 실질적인 지표는 사용자들이 실제로 가치 있다고 인정하는 서버를 쉽게 식별할 수 있게 도와줍니다.
- **MCP 생태계의 독보적 위치:** **Smithery**의 성공의 열쇠는 개방성과 커뮤니티 중심 접근법에 있습니다.

6. MCP 실습(2): 가져다 쓰는 MCP서버

2) 카카오 PlayMCP

- 카카오에서는 PlayMCP 베타 오픈(2025.8.13.)
- PlayMCP는 웹 브라우저에서 바로 사용할 수 있는 MCP 플랫폼으로 카카오 계정만 있으면 누구나 참여할 수 있으며, 개발자는 자신이 만든 MCP 서버를 등록하고 테스트 가능



6. MCP 실습(2): 가져다 쓰는 MCP서버

2) 카카오 PlayMCP

- 사용방법 (별도의 클라이언트 설치가 필요하지 않음)

1. <https://playmcp.kakao.com/> 접속
2. 카카오 계정으로 로그인 (본인 인증 필요)
3. 원하는 MCP 서버 선택
4. 'AI 채팅' 클릭
5. 웹 채팅창에서 대화 시작

○ 카카오맵 MCP 서버 사용예

- 카카오맵 MCP가 지원하는 도구:

- **GetWalkDirections**: 도보 경로 안내
- **GetBikeDirections**: 자전거 경로 안내
- **GetPublicTransitDirections**: 대중교통 경로 안내
- **SearchPlaceByKeywordOpen**: 키워드 기반 장소 검색

6. MCP 실습(2): 가져다 쓰는 MCP서버

2) 카카오 PlayMCP

- 사용예:

The screenshot displays the PlayMCP web interface in a browser window. The main page is titled '도구함' (Toolbox) and shows a list of installed MCPs. One MCP, '카카오맵' (KakaoMap), is highlighted with a toggle switch set to 'ON'. A sidebar on the right shows a list of installed MCPs, including '카카오맵'.

Overlaid on the right is an AI chat window titled 'AI 채팅'. The chat history shows a question: '광화문에서 서울역까지 도보로 얼마나 걸려?' (How long does it take to walk from Gwanghwamun to Seoul Station?). The AI response is: '광화문에서 서울역까지 도보로 약 2.7km이며, 약 45분 정도 소요됩니다. 상세 경로는 카카오맵에서 확인하세요.' (It is about 2.7km and takes about 45 minutes to walk from Gwanghwamun to Seoul Station. Check the detailed route on KakaoMap).

The chat window also shows a 'TOOL 호출' (Tool Call) section with a table of the request and response:

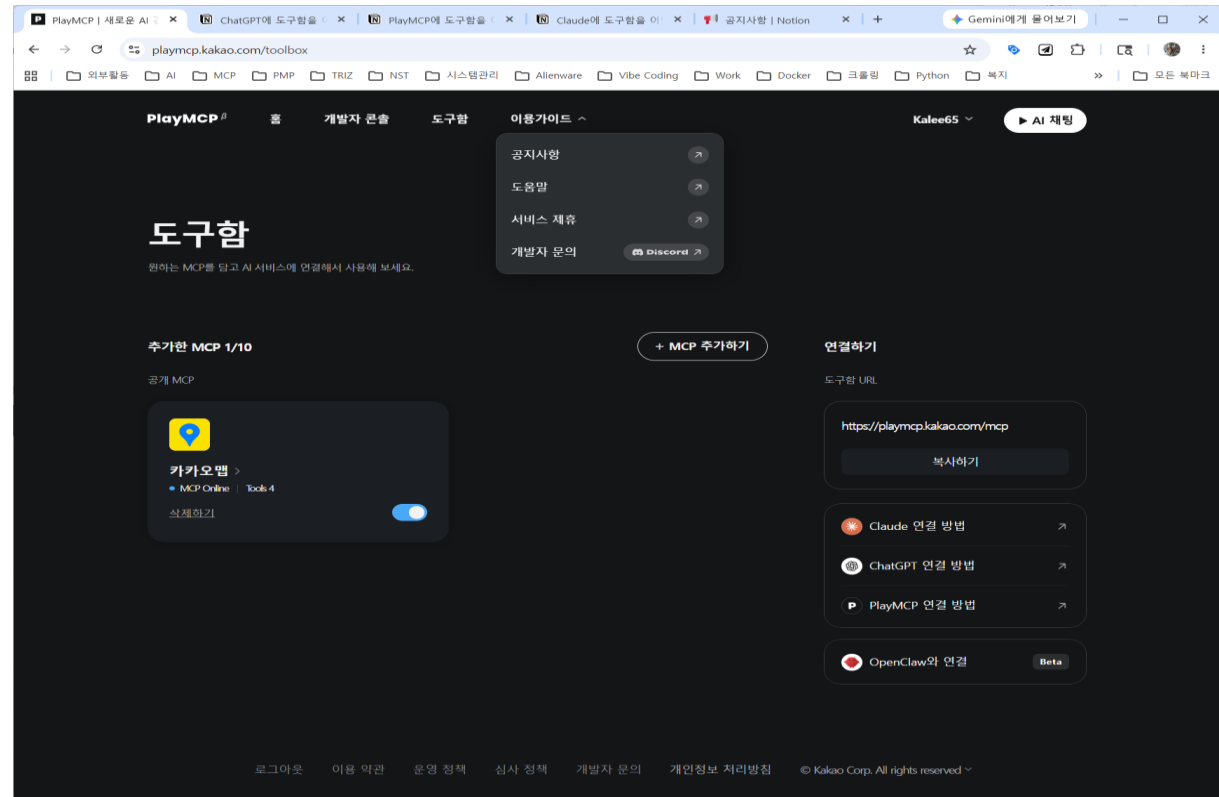
Request	Response
<pre>{ "method": "tools/call", "params": { "name": "GetWalkDirections", "arguments": { "origin": "광화문", "destination": "서울역", "routeMode": "RECOMMENDATION" } }}</pre>	

At the bottom of the chat window, there is a text input field with the placeholder '질문을 입력해 주세요.' (Please enter a question.) and a button labeled '도구함 1' (Toolbox 1).

6. MCP 실습(2): 가져다 쓰는 MCP서버

2) 카카오 PlayMCP

- 도구함에서 원하는 MCP 담고 AI 채팅 클릭후 프롬프트 입력하면 결과 출력됨
- MCP 서버 등록 과정
 1. 카카오 계정으로 가입
 2. Remote MCP Server URL과 설명 제출
 3. 기능 검증 및 보안 점검
 4. 승인 후 사용자에게 공개
- 이용가이드 및 개발자 문의(discord 이용)



3) 엑셀 MCP 서버 가져와서 활용하기

- 소스 코드 버전: 에러 발생시 의존성 패키지까지 설치해 주어야 정상 작동
- 자동 실행 버전: 별도의 설치나 설정 없이 실행 파일 다운로드 받아 설치후 연결 확인
- 다음 사이트에서 pyhub.mcptools-windows-v0.9.5.zip 다운로드
<https://github.com/pyhub-kr/pyhub-mcptools/releases>
- C:\₩ 아래에 mcptools 폴더를 만들고 압축 해제
- pyhub.mcptools 폴더 아래의 pyhub.mcptools.exe 파일 실행
- 설정은 claude_desktop_config.json 파일을 열고 다음과 같이 수정

```
{  
  "mcpServers": {  
    "pyhub.mcptools": {  
      "command": "C:\\₩\\mcptools\\₩\\pyhub.mcptools\\₩\\pyhub.mcptools.exe",  
      "args": [  
        "run",  
        "stdio"  
      ]  
    }  
  }  
}
```

3) 엑셀 MCP 서버 가져와서 활용하기

- 클로드를 다시 실행해서 설정 > 개발자 로 들어가서 pyhub.mcptools 가 연결되었는지 확인
- [주의사항] 사용 전 반드시 엑셀이 미리 실행되어 있어야 도구가 열려 있는 엑셀 파일을 자동으로 인식
- 본 도구의 장점은 엑셀 파일을 실시간으로 읽고 수정 가능하다는 점, 즉 엑셀 프로그램과 실시간으로 직접 통신 가능하며 이를 통해 실시간 편집이 가능
- <https://smithery.ai> 에 접속하면 많은 MCP가 등록되어 있으므로 다운로드 받아 자유로운 활용이 가능하나 보안 문제가 발생할 수 있으니 활용에 각별한 주의가 필요

4) 아래한글(HWP) 파일 읽고 쓰기

- 주요 HWP MCP 프로젝트 비교 (2026.01. 기준)

구분	jkf87/hwp-mcp	skerishKang/MCP_HWP_Limone	m1ns2o/hwp-mcp-go
언어/환경	Python (Windows 전용)	Python (Windows 전용)	Go (Windows 전용)
핵심 기술	pywin32 (COM API)	pywin32 (COM API)	mark3labs/mcp-go (COM API)
주요 강점	안정적인 표(Table) 처리 및 일괄 작업(Batch) 기능	고급 서식 제어 (이미지 삽입, 스타일, 페이지 설정)	고성능/경량화. 단일 실행 파일 배포 가능
제어 범위	기본 문서 생성, 텍스트 삽입, 복합 표 생성	텍스트 위치 지정, 이미지/도형 삽입, 하이퍼링크	문서 생성, 텍스트 및 기본 표 제어 (jkf87의 Go 포팅판)
문서 형식	HWP, HWPX 모두 지원	HWP, HWPX 모두 지원	HWP, HWPX 모두 지원

4) 아래한글(HWP) 파일 읽고 쓰기

❖ 기술적 특징 비교

각 프로젝트는 한글(HWP) 자동화의 접근 방식에 따라 다음과 같은 기술적 차이를 보임

1. 작동 방식의 공통점: Windows COM API (HwpObject)

위의 모든 프로젝트는 윈도우 환경에 설치된 한글 프로그램을 직접 제어하는 **COM(Component Object Model)** 방식을 사용

장점: 한글 프로그램이 지원하는 모든 기능을 100% 활용할 수 있어, 표의 셀 병합이나 복잡한 서식 적용이 매우 정확

단점: 반드시 Windows OS여야 하며, 배경에 한글(hwp.exe)이 실행되고 있어야 함

2. 프로젝트별 상세 차이점

- **jkf87/hwp-mcp (표 특화형):** AI가 가장 어려워하는 '표 데이터 입력' 로직이 정교합니다. hwp_batch_operations를 통해 여러 명령을 한 번에 처리하여 AI와의 통신 횟수를 줄이는 효율적인 구조를 지님
- **MCP_HWP_Limone (고급 편집형):** 단순 텍스트 입력을 넘어 이미지 삽입 (insert_image), 머리글/바닥글 설정, 목차 생성 등 실제 전문 보고서를 작성할 때 필요한 세부 도구(Tools) 제공
- **hwp-mcp-go (성능 최적화형):** Python 설치 없이 실행 파일 하나로 서버를 띄울 수 있어 배포가 간편하며, Go 언어 특유의 빠른 응답 속도

3. 추천 활용 시나리오

- 데이터 보고서 생성 시: 표 제어가 강력한 **jkf87/hwp-mcp**를 추천
- 이미지/도형 포함 복잡한 문서 작성 시: **Limone** 서버가 유리
- 가벼운 환경에서 빠른 연동을 원할 시: **hwp-mcp-go**를 고려해 볼 것
- 본 교육과정에서는 널리 사용되고 있는 jkf87/hwp-mcp 방식을 중심으로 설명
- 좀 더 자세한 사용 방법을 알고 싶으면 개발자가 직접 시연한 "MCP로 HWP 자동화" 유튜브 영상 <https://www.youtube.com/watch?v=SE1qJ-zzm8o> 참조

4) 아래한글(HWP) 파일 읽고 쓰기

- ❖ 아래아 한글(HWP) 파일은 OLE(Object Linking and Embedding) 구조를 사용하며, 이는 파일 하나가 단순한 텍스트 덩치가 아니라 파일안에 구축된 파일 시스템이라는 의미

1. OLE 파일 구조의 핵심 개념: "파일 속의 폴더"

- 일반적인 텍스트 파일(.txt)이 한 줄로 쭉 이어진 데이터라면, HWP는 윈도우의 '폴더(Directory)'와 '파일(File)' 구조를 파일 하나에 담고 있습니다.
- 스토리지(Storage): 윈도우의 폴더에 해당합니다. 관련 데이터를 묶는 역할을 합니다.
- 스트림(Stream): 윈도우의 파일에 해당합니다. 실제 데이터(텍스트, 이미지, 서식 정보 등)가 저장되는 공간입니다.

4) 아래한글(HWP) 파일 읽고 쓰기

2. HWP 파일 내부의 주요 구성 요소

- HWP 파일을 OLE 구조 분석 도구(예: SSView, OLEViewer)로 열어보면 다음과 같은 구조를 볼 수 있습니다.

이름	유형	역할
FileHeader	스트림	HWP 파일임을 증명하는 시그니처와 버전 정보 저장
DocInfo	스트림	문서에 사용된 글꼴, 문단 모양, 스타일, 색상 정보 등 집합
BodyText	스토리지	실제 문서의 본문 내용이 담긴 구역(Section)들이 저장된 폴더
BinData	스토리지	문서에 삽입된 이미지, 멀티미디어 등 이진(Binary) 파일 저장
SummaryInformation	스트림	문서 제목, 지은이, 날짜 등 문서 속성 정보

4) 아래한글(HWP) 파일 읽고 쓰기

3. 왜 이런 구조를 사용할까?

- 1) **효율적인 관리:** 문서에 수백 장의 고화질 이미지가 포함되어 있어도, 특정 페이지의 텍스트만 읽고 싶을 때 파일 전체를 메모리에 올릴 필요 없이 해당 '스트림'만 찾아 읽으면 됩니다.
- 2) **데이터 무결성:** 텍스트 데이터와 이미지 데이터가 물리적으로 분리된 구역에 저장되므로, 한쪽 데이터가 손상되어도 다른 쪽 데이터를 복구하기 용이합니다.
- 3) **확장성:** 새로운 기능(예: 새로운 표 스타일, 차트)이 추가될 때 기존 구조를 깨지 않고 새로운 '스트림'만 추가하면 되므로 하위 호환성 유지에 유리합니다.

4. 기술적 주의사항: 압축과 암호화

HWP의 OLE 구조 내 각 스트림 데이터는 보통 zlib으로 압축되어 있습니다. 따라서 파일을 단순히 메모장으로 열면 글자가 깨져 보입니다. 또한, 문서 암호가 설정된 경우 각 스트림이 암호화되어 저장되므로, 외부 라이브러리로 분석할 때는 반드시 압축 해제와 복호화 과정이 선행되어야 합니다.

참고: 최신 한글 표준인 HWPX는 OLE 구조가 아닌 "XML 기반의 ZIP 압축 방식(OOXML)"을 사용합니다. 이는 웹 환경과 데이터 분석에 더 적합하도록 현대화된 구조입니다.

4) 아래한글(HWP) 파일 읽고 쓰기

[실습]

- 한글 파일 처리를 위해서는 `olefile`, `markdown`, `bs4` 라이브러리가 필요
`pip install olefile markdown beautifulsoup4`
 - `olefile`: 복합문서 파일형식을 처리하는 라이브러리
 - `markdown`: 마크다운 형식을 HTML로 변환하는 라이브러리
 - `beautifulsoup4`: HTML 구조를 파싱하고 처리하는 라이브러리
- 본 과정에서는 `hwp` 파일을 읽고 조회하는 기능 추가와 `md` 파일을 읽어 `hwp`로 저장하는 기능을 구현하고 확인하는 실습을 진행
- `server.py`은 `sample.md` 파일을 `hwp` 파일로 변환하여 저장하는데 이를 위하여 `gen.py` 기능을 이용하여 변환
- `server.py` 파일과 `gen.py` 파일을 같은 폴더 (`C:\wproject\my_mcp`)에 저장 후 실행

4) 아래한글(HWP) 파일 읽고 쓰기

프롬프트 입력

"C:/test 폴더에 있는 test.hwp 파일을 읽어줘

다음 "" 안의 내용을 c:/test 폴더에 있는
report.hwp로 저장해줘

"# 월간보고서

프로젝트 현황

* 개발 진행률: 75%

* 주요 이슈: 없음

* 다음 마일스톤: 테스트 단계

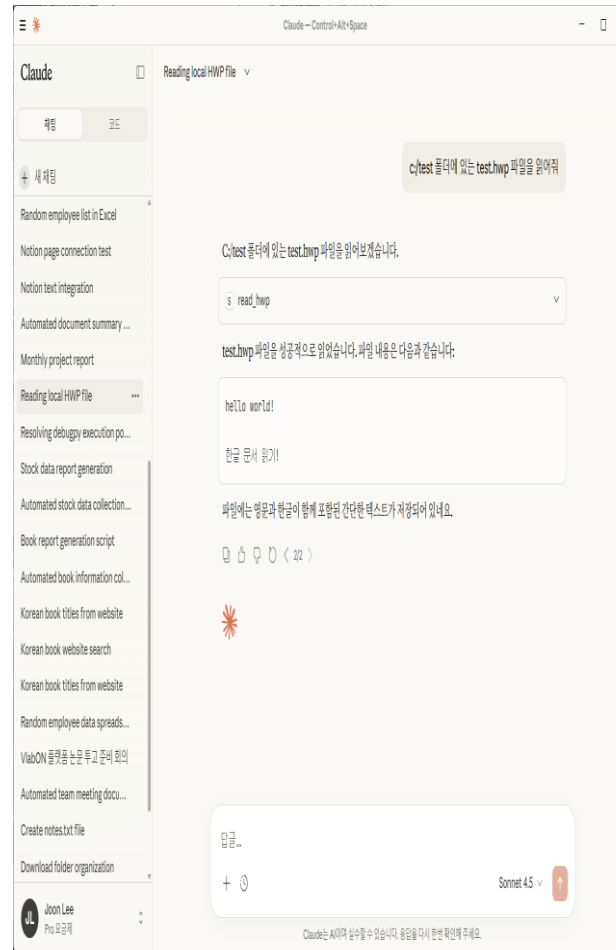
예산 사용 현황

1. 인건비: 500만원

2. 경비비: 200만원

3. 기타: 50만원

"



5) 기획 보고서 작성

- 스미더리에서 “Sequential Thinking”을 검색하고 @xinzhongyouhai 제작 Sequential Thinking Tools 를 선택하고 클로드 데스크탑을 눌러 설치 명령어를 받고 명령어를 실행
 - Sequential Thinking MCP는 어떤 일을 하는데 절차가 잘 생각나지 않거나 별다른 아이디어가 없을 때 활용하기 좋은 도구
 - 다음 프롬프트를 실행하여 결과를 비교해 볼 것
(프롬프트 1) 의자를 팔고 싶은데 전략을 어떻게 세워야 할 지 알려줘
(프롬프트 2) Sequential Thinking MCP의 도움을 받아서 의자를 팔고 싶은데 전략을 어떻게 세워야 할 지 알려줘
 - Sequential Thinking MCP 가 제공하는 핵심 값
 - ✓ thoughtNumber: 현재 생각하고 있는 단계
 - ✓ totalThoughts: 생각의 총 단계
 - ✓ revisesThought: 다시 고려하고 있는 생각
 - ✓ branchFromThought: 현재 생각으로부터 뻗어나온 생각
- (추가 질문 1) 40대를 대상으로 의자를 팔고 싶어. 그러면 의자의 종류, 브랜드, 상태, 특별기능, 디자인은 어떻게 해야 할 지 알려줘
- (추가 질문 2) Sequential Thinking MCP를 활용해서 40대를 위한 의자에 대한 상품 기획 보고서를 마크다운으로 작성해서 바탕화면 위치에 저장해 줘
- (추가 질문 3) 이 파일을 hwpx 파일로 변환해 줘. 이때 다음 내용에 대한 수정 작업을 마치고 hwpx로 변환해 줘. 표로 정리하면 보기 좋은 내용은 표로 정리해 줄 것

6) MBTI 테스트 앱 만들기

(프롬프트 1) Sequential Thinking MCP를 활용해서 MZ 세대가 좋아할 만한 주제를 하나 고르고 해당 주제로 20개의 MBTI 검사를 할 수 있는 질문을 생성해 줘

(프롬프트 2) Sequential Thinking MCP를 활용해서 지금 만든 각 질문에 부여할 점수를 설계해 줘. 질문의 정도에 따라 다른 점수를 부여해 줘

(프롬프트 3) Sequential Thinking MCP를 활용해서 만든 질문을 바탕으로 한 MBTI 앱을 간단한 방법으로 만들어 줘

* 중간에 오류가 발생할 경우 오류 메시지를 복사하여 채팅창에 붙여 넣고 계속 진행

- 생성된 MBTI 앱을 HTML로 다운로드

- 웹 배포를 위해 <https://v0.app/> 사이트에 접속하여 회원가입한 후 다운로드된 앱을 첨부파일로 넣어서 다음 프롬프트 입력

(프롬프트 4) 첨부파일을 참고해서 화면을 유지한 채로 MBTI 앱을 만들어줘

- [Publish → Deploy to Production]을 누르면 배포 완료되며 [Visit Site] 버튼이 나타남. 버튼을 누르면 배포한 앱이 보임

MCP 이해와 활용(실습 II)

오후 수업 (12:30~16:50)

0-2 강의 진행

❖ 오전

- MCP의 이론 및 관련 기술에 대한 소개
- 실습 환경 구축과 에러시 대응 방안, 간단한 실습

❖ 오후

- 외부에서 MCP 서버 가져다 쓰는 방식 실습
- MCP 코드 작성 및 직접 설정 방식 위주의 예제 실습
- 반복업무 자동화 및 직무별 활용사례



프롬프트의 4가지 핵심 구성요소

- 페르소나(Persona): 역할과 정체성 부여하기
 - ✓ 사용 예: 너는 10년 경력의 개발자야. 너는 초등학교 선생님이야.
- 작업(Task): 구체적인 행동을 지시하기
 - ✓ 사용 예: 1단계로 애플리케이션 구조를 설계하고 2단계로 실행 계획을 작성하고 3단계로 구현해 줘
- 맥락(context): 배경 설명하기
 - ✓ 사용 예: 기존 시스템의 영향을 받지 않도록 별도로 추가 개발해줘
- 형식(Format): 출력 형태 지정하기
 - ✓ 사용 예: 체크 리스트 형식으로 작성해 줘. PPT 슬라이드형태로 작성해줘



효율적인 프롬프트 작성법

- 명확하고 구체적으로 지시하기
 - ✓ 사용 예: 파이썬에 대해 설명해 줘 =>
 - ✓ 초등학생의 관점에서 프로그래밍 언어 파이썬에 대해 설명해줘
- 예시를 들어 설명하기
 - ✓ 사용 예: 다음 코드를 참조해서 코드를 작성해 줘
- 참고자료나 배경 지식에 도움이 될 정보가 있다면 제공하기
 - ✓ 사용 예: 첨부 문서를 참조해서 4개의 핵심 문장으로 요약해줘
- 답변형식과 범위를 지정하기
 - ✓ 사용 예: 답변 형식을 md 형식으로 제시하되 기술적인 내용은 제외하고 일반인도 이해 가능하도록 200자 내외로 작성해줘

RTF 공식

- R(Role): AI 또는 프롬프트의 역할을 명확히 정의
- T(Task): AI가 성취할 필요가 있는 업무(의무)를 간략히 기술
- F(Format): AI가 생성한 응답의 바람직한 형식(Format) 또는 구조(Structure)를 구체화
- 예시:
 - ✓ 당신은 15년차 소프트웨어 개발과 프로젝트 관리 경험을 지니고 있다.
 - ✓ 당신의 과업은 현재 노후화된 하드웨어 장비의 교체에 따른 데이터 이전을 기존 수행하는 업무의 중단없이 주어진 예산 범위내에서 정해진 기한내에 성공적으로 수행 가능하도록 잠재적인 위험 요인을 식별하는 것이다.
 - ✓ 테이블 형식으로 프로젝트의 잠재적 위험요인을 찾아서 작성해 줘



CREATE 공식

- C (Character): 당신은 15년차 소프트웨어 개발 및 프로젝트 관리자이다.
- R(Request): 노후화된 하드웨어 교체에 따른 잠재적 위험 요인을 식별해야 한다.
- E(Example):
 - ✓ 일정 지연과 관련된 위험을 식별
 - ✓ 예산 초과와 관련된 위험을 식별
 - ✓ 데이터 이전에 따른 위험을 식별
 - ✓ 보안 관련 위험을 식별
 - ✓ 소프트웨어의 호환성과 관련된 위험 식별
- A(Adjustments and constraints): 따라야 하는 규범과 표준 확인과 기존 하드웨어 관리 운영 관행과 지침
- T(Types of output): 위험 목록, 위험 평가 보고서, 위험 경감 계획
- E(Evaluation and steps):
 - ✓ 타임라인: 프로젝트 계획 단계에서 위험 평가 완료 확인
 - ✓ 품질: 주요 위험 요인의 식별, 평가 확인
 - ✓ 이해관계자의 만족(stakeholder satisfaction): 주요 이해관계자의 위험 요구사항 식별 조치

참고: PMI 2024, PromptEngineeringWorkbook.pdf

7. MCP 실습(3): MCP 코드 작성 및 직접 설정 방식

1) FastMCP 개념과 활용

- FastMCP는 MCP 서버와 클라이언트를 구축하기 위한 파이썬 프레임워크로, 현재 MCP 생태계에서 사실상의 표준(de-factor standard) 입지를 확보
- **FastMCP의 역사:** 초기 버전인 FastMCP 1.0은 2024년 공식 MCP Python SDK에 통합되었고 FastMCP 2.0은 공식 SDK에서 분기되어 단순한 프로토콜 구현을 넘어 프로덕션 환경에 필요한 포괄적인 기능을 제공하는 독립적인 프로젝트로 발전
- **FastMCP의 설계 철학:** “빠르고(Fast), 파이썬스럽고(Pythonic), 완전하다(Complete)”로 요약되며, 이는 개발자가 최소한의 코드로 직관적이고 명확하게 MCP서버를 구축할 수 있도록 지원하며, 동시에 배포, 인증, 미들웨어, 테스트 도구 등 실제 애플리케이션 개발에 필요한 모든 것을 제공할 것이라는 의지를 포함하고 있음

2일차 오후

7. MCP 실습(3): MCP 코드 작성 및 직접 설정 방식

1) FastMCP 개념과 활용

○ FastMCP 서버 구축단계

1단계: 환경설정

프로젝트를 위한 가상환경을 만들고
FastMCP를 설치: 'uv' 또는 'pip'을 사용

```
# uv 사용 시
uv venv
source .venv/bin/activate
uv add "mcp[cli]"

# pip 사용 시
python -m venv .venv
source .venv/bin/activate
pip install "mcp[cli]"
```

2단계: 기본 서버 생성

프로젝트 폴더에 `my\_server.py`라는 파일을 생성하고,
`FastMCP` 클래스를 임포트하여 서버 인스턴스를 생성.
서버 이름은 클라이언트 UI에 표시

```
# my_server.py
from fastmcp import FastMCP

mcp = FastMCP("My First Server 🚀")
```

7. MCP 실습(3): MCP 코드 작성 및 직접 설정 방식

1) FastMCP 개념과 활용

○ FastMCP 서버 구축단계

3단계: Tool 추가

두 숫자를 더하는 간단한 파이썬 함수를 작성하고, `@mcp.tool` 데코레이터를 붙여주기만 하면 이 함수는 MCP Tool로 등록됨. FastMCP는 함수의 타입 어노테이션 (`a: int, b: int`)과 독스트링(`"""Add two numbers"""`)을 자동으로 분석하여 클라이언트에게 필요한 스키마와 설명을 생성

```
# my_server.py (이어서)

@mcp.tool
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b
```

4단계: 서버 실행 및 테스트

서버를 실행하는 가장 간단한 방법은 파일 끝에 실행 블록을 추가하는 것임. `if \_\_name\_\_ == "\_\_main\_\_"` 블록은 스크립트로 직접 실행될 때만 서버가 작동하도록 보장하여, 다른 파일에서 임포트될 때 원치 않는 실행을 방지

```
# my_server.py (이어서)

if __name__ == "__main__":
    # stdio: 로컬 클라이언트와 통신 (기본값)
    # http: 원격 클라이언트와 통신
    mcp.run(transport="http")
```

7. MCP 실습(3): MCP 코드 작성 및 직접 설정 방식

1) FastMCP 개념과 활용

○ FastMCP 서버 구축단계

- 터미널에서 다음 명령어로 서버를 실행할 수 있음

```
python my_server.py
```

- 또는 FastMCP CLI를 사용하여 실행할 수도 있는데, 이 방법은 코드 내의 'mcp.run()' 설정을 무시하고 CLI 옵션을 우선 적용함

```
fastmcp run my_server.py --transport http
```

7. MCP 실습(3): MCP 코드 작성 및 직접 설정 방식

1) FastMCP 개념과 활용

○ FastMCP 서버 구축단계

5단계: 클라이언트에서 서버 호출하기

다른 파이썬 스크립트에서 `FastMCP` 클라이언트를 사용하여 실행 중인 서버에 연결하고 `add` Tool을 호출하면, FastMCP 클라이언트는 비동기(asynchronous) 방식으로 작동하므로 `asyncio`를 사용해야 합니다.

```
# my_client.py
import asyncio
from fastmcp.client import FastMCP

async def main():
    # HTTP 전송 방식으로 실행된 서버에 연결
    async with FastMCP.create("http://127.0.0.1:8000") as client:
        # 'add' tool 호출
        result = await client.tools.add(a=10, b=5)
        print(f"Result of 10 + 5 is: {result}") # 출력: Result of 10 + 5 is: 15

if __name__ == "__main__":
    asyncio.run(main())
```

7. MCP 실습(3): MCP 코드 작성 및 직접 설정 방식

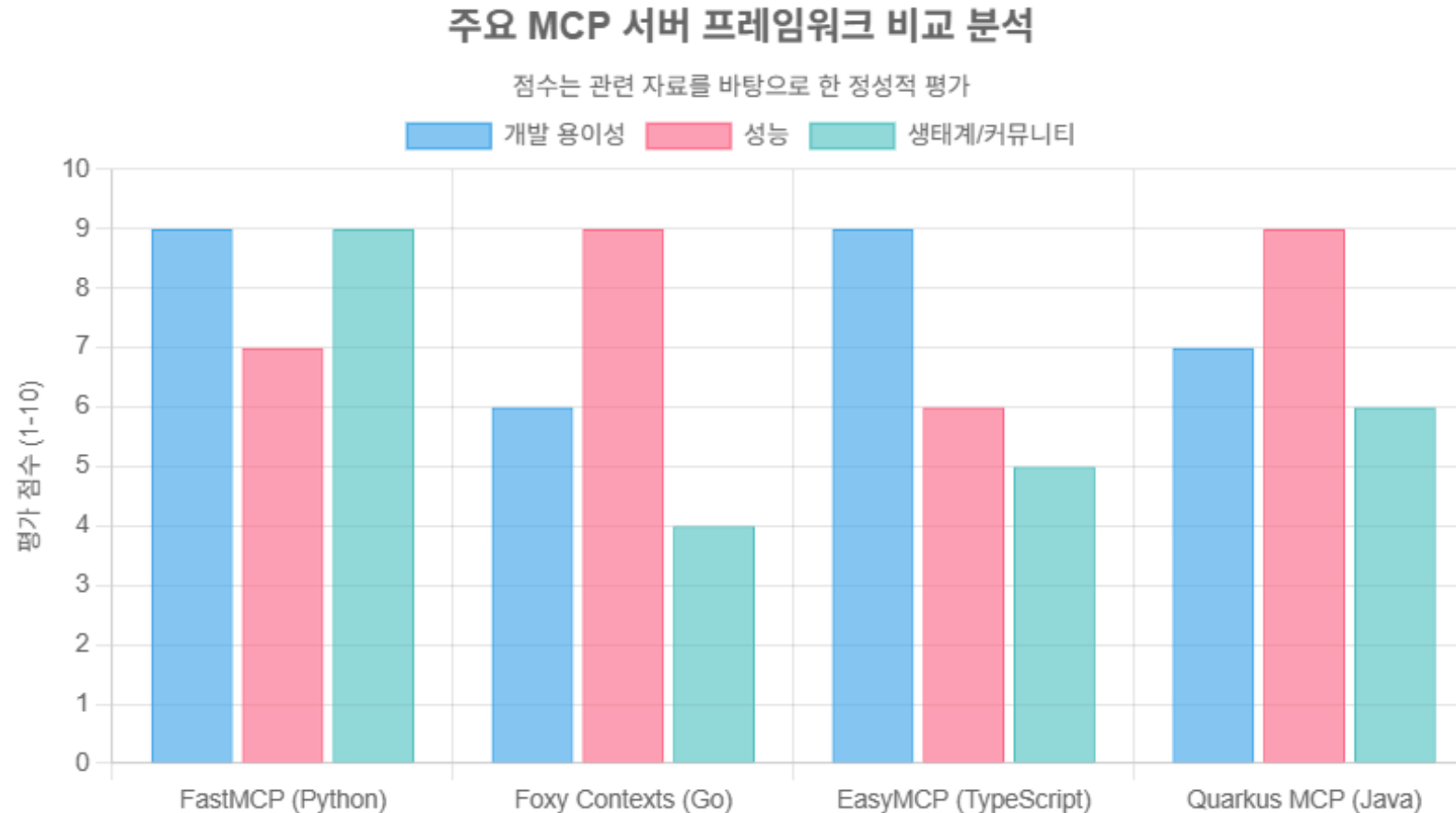
1) FastMCP 개념과 활용

○ 성능 비교분석: FastMCP(Python) vs. Foxy Contexts(Go)

- MCP 서버 프레임워크 대표 주자로는 파이썬 기반의 FastMCP와 Go 기반의 Foxy Contexts가 있음. 컴파일 언어인 Go는 인터프리터 언어인 파이썬보다 원시 연산 성능이 뛰어나 동시성 처리와 낮은 지연 시간이 중요한 고성능 백엔드 서비스에 강점을 지님.
- 하지만 AI 에이전트의 워크로드는 대부분 CPU 집약적인 계산보다는 외부 API 호출이나 데이터베이스 조회와 같은 I/O 바운드(I/O-bound) 작업이 주를 이룸. 이런 환경에서는 파이썬의 강력한 비동기(async/await) 지원이 중요함. FastMCP는 `async` 함수를 완벽하게 지원하여 I/O 대기 시간 동안 다른 요청을 효율적으로 처리할 수 있으므로, 대부분의 MCP 서버 시나리오에서 충분히 높은 성능을 제공함. 또한, 방대한 라이브러리 생태계와 빠른 개발 속도는 파이썬과 FastMCP가 가진 명백한 강점임

7. MCP 실습(3): MCP 코드 작성 및 직접 설정 방식

1) FastMCP 개념과 활용



2) 웹 데이터 가져오기

- 크롬에서 웹 페이지 구성요소를 확인하는 방법: 크롬 브라우저 웹페이지에서 마우스 우클릭하고 "검사" 선택 또는 F12 키를 누르면 개발자 도구가 열림. 상단의 Elements 탭을 통해 HTML 구조 확인 가능
- 웹 크롤링을 위해서는 다음 라이브러리 설치가 필요
- requests: 웹 페이지를 요청하고 가져오는 라이브러리
- BeautifulSoup: HTML을 분석하여 필요한 정보를 추출하는 라이브러리
- pandas: 데이터를 표 형태로 관리하고 엑셀 파일로 저장하는 라이브러리
- 터미널을 열고 다음 명령어 실행하여 필요한 라이브러리 설치

```
pip install pandas requests bs4
```

[예제 1] 책 정보로부터 리스트 출력

- MCP코드.md 파일의 해당 코드를 crawler_1.py 으로 저장하고 python crawler_1.py 실행
- 위니브 책 정보 페이지로부터 책 제목 정보를 추출하여 리스트를 출력

[예제 2] 주가지수 서비스로부터 일일 증시 현황 확인

- 해당 코드를 crawler_2.py 으로 작성하여 저장하고 python crawler_2.py 으로 실행
- 에러가 나올 경우 pip install lxml 설치

3) 도서 정보 자동 수집 및 마크다운 리포트 작성

- 프롬프트 작성 (MCP 프롬프트.md)
- python book_collector.py 실행하고 클로드에서 collect_book_info 호출하여 실행

[주의사항] 웹 크롤링을 하려면 대상 웹사이트가 크롤링을 허용하는지 반드시 확인이 필요함. 해당 페이지 뒤에 /robots.txt를 입력하여 크롤링을 허용하는지 여부를 확인할 것. 구글의 경우 <https://www.google.com/robots.txt> 에서 확인 가능



4) 주가 데이터 자동 수집 및 분석 리포트 생성하기

The screenshot displays the Claude AI web interface. On the left is a sidebar with a list of recent prompts, including 'Automated stock data collection script'. The main area shows the content of this script, which is written in Korean. The script describes an MCP (Model Context Protocol) function for collecting and analyzing stock data from the '위니브' (WineB) website. It lists four main features: data collection, information extraction, report generation, and automatic saving. The report generation feature specifies that the report will be a markdown file with four sections: company key indicators, 5-day price trends, company comparison analysis, and trading volume trends. The script also includes a usage method section with a Python code snippet for calling the `collect_stock_report()` function and a bash command for installing the necessary dependencies (`requests`, `beautifulsoup4`, and `mcp`). At the bottom, there is a text input field with a plus icon and a 'Sonnet 4.5' model selector.

Claude — Control+Alt+Space

Automated stock data collection script

위니브 주식 정보를 자동으로 수집하고 분석 리포트를 생성하는 MCP 함수를 만들어드리겠습니다.

위니브 주식 데이터 수집 및 분석 MCP
코드

이 MCP 함수의 주요 특징을 설명해드리겠습니다:

주요 기능:

1. 데이터 수집 - `collect_stock_report()` 함수가 위니브 주식 정보 페이지에서 HTML을 가져와 BeautifulSoup으로 파싱합니다.
2. 정보 추출 - 다음 정보들을 자동으로 추출합니다:
 - 회사별 시가총액, 현재가, 52주 최고/최저가
 - 일일 주가 데이터 (날짜, 종가, 전일비, 시가, 고가, 저가, 거래량)
3. 마크다운 리포트 생성 - 4개 섹션으로 구성된 리포트를 자동 생성합니다:
 - 회사별 주요 지표 요약표
 - 최근 5일간 주가 동향
 - 회사간 비교 분석표
 - 거래량 및 변동률 순위
4. 자동 저장 - `c:/test/reports/` 폴더에 `stock_report_YYYYMMDD.md` 형식으로 저장합니다.

사용 방법:

```
python
# MCP 클라이언트에서 이 도구를 호출하면:
collect_stock_report()

# 매일 자동으로 주식 데이터를 수집하고 분석 리포트를 생성합니다.
```

필요한 라이브러리:

```
bash
pip install requests beautifulsoup4 mcp
```

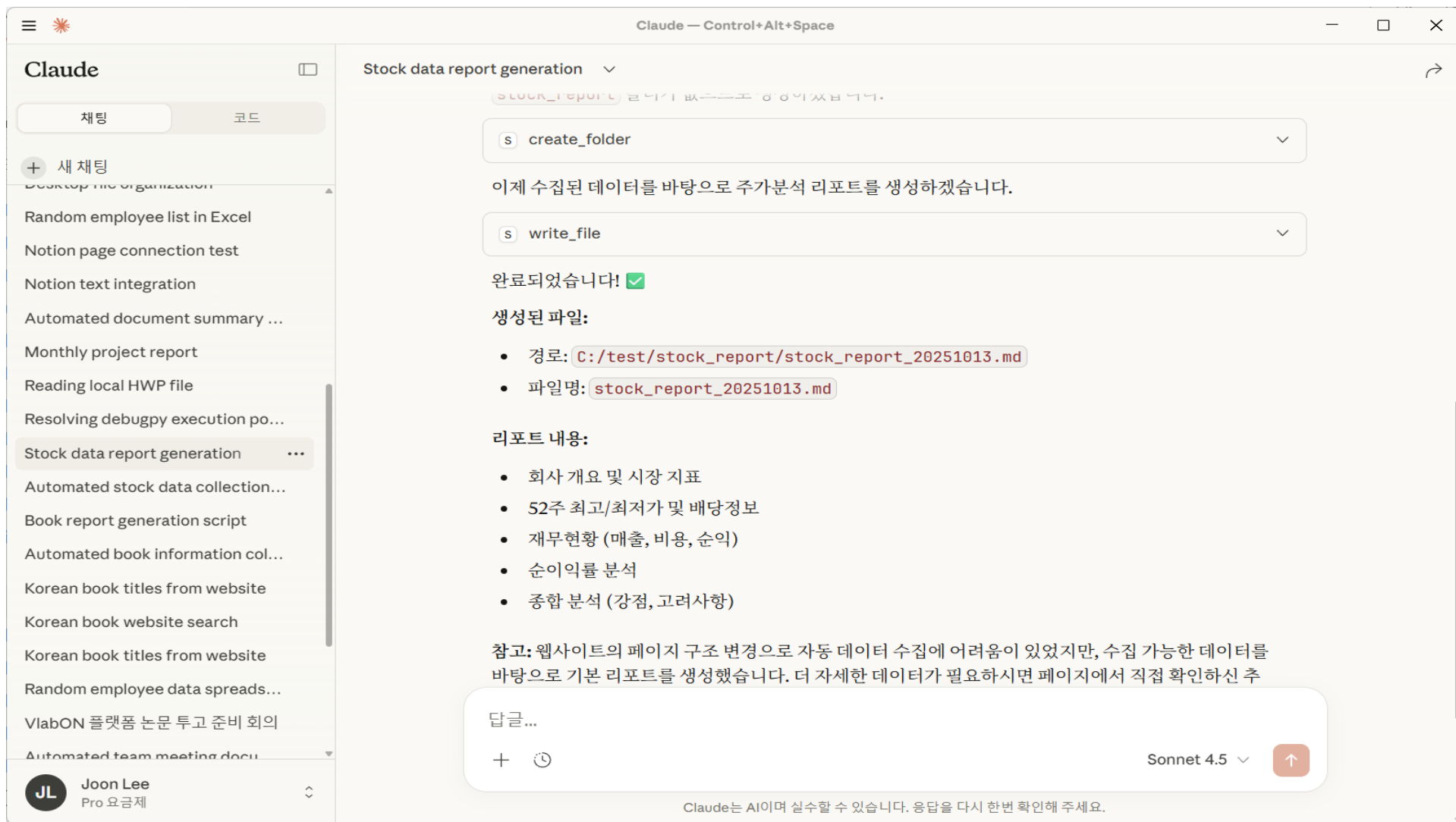
이제 수작업 없이 매일 자동으로 주식 분석 리포트를 생성할 수 있습니다! 🥳

답글...

+ 🕒 Sonnet 4.5

Claude는 AI이며 실수할 수 있습니다. 응답을 다시 한번 확인해 주세요.

○ 주가 분석 리포트 자동 생성하기



5) 공공기관 문서 검토 업무 자동화

- 프롬프트 작성
- 클로드 코드 자동 생성
- 이러한 자동화도구를 통해 대량의 공공 문서 검토 업무의 효율성과 생산성을 대폭 향상시킬 수 있으며 분석 리포트를 신속하게 생성할 수 있음
- 다만 생성된 리포트는 LLM에 의해 생성된 결과인 만큼 충분한 검토를 거쳐 활용할 필요가 있음

6) 제안서 초안 자동 생성 실습

- 프롬프트를 입력하고 클로드가 생성한 코드를 활용하여 .md 파일을 생성하고 생성된 .md 파일을 .hwp로 저장



잘 작동하는지 확인하고 싶으신가요? NTIS 사이트 또는 조달청 사이트에 접속하셔서 샘플 파일을 다운로드 받거나 클로드에게 테스트를 위한 샘플 작성을 요청하신 후 test 폴더에 저장하고 클로드가 생성해 준 MCP 함수를 활용해서 생성된 hwp 파일을 확인해 보세요

7) 엑셀 파일 읽고 쓰기

- 아래한글(hwp) MCP서버와는 달리 엑셀 파일을 다루는 MCP는 다수 존재
- 클로드에서 엑셀을 다루기 위해서는 다음 라이브러리 설치(설치형 실습)
pip install pandas xlsxwriter openpyxl
- [실습 1] excel_pandas.py 코드 작성
- [실습 2] Excel_xlsxwriter_1.py 코드 작성
- [실습 3] Excel_xlsxwriter_2.py 코드 작성
- [실습 4] server.py 코드 작성
- MCP 설정파일(claude_desktop_config.json)을
- 터미널을 열고 다음 명령 입력
mcp install server.py
- 의미는 MCP로 server.py 파일을 등록하려고 하니 설정파일에 server.py 파일을 등록해 줘
- 에러시 pip install uv 명령어로 uv 설치 후 재 실행
- 설정파일을 다시 열어서 다음과 같은 내용이 등록되어 있으면 성공

```
{  
  "mcpServers": {  
    "mcp_project": {  
      "command": "uv",  
      "args": [  
        "run",  
        "--with",  
        "mcp[cli]",  
        "mcp",  
        "run",  
        "C:\\\\test\\\\server.py"  
      ]  
    }  
  }  
}
```

7) 엑셀 파일 읽고 쓰기

- 설정 완료 후 파일 > 종료 선택하고 클로드 재 실행

- 다음 프롬프트 입력

* `create_excel_with_formatting` 함수 활용 예

10명의 직원 정보가 담긴 엑셀 파일을 만들어 줘, 나이, 부서, 입사일 정보가 포함되어야 하고 데이터는 랜덤하게 작성하고 직원정보.xlsx 로 저장해 줘

* `read_excel` 함수 활용 예

직원정보.xlsx 파일의 내용을 읽어줘

* `append_to_excel` 함수 활용 예

엑셀 파일에 10명의 새로운 직원 정보를 추가해 줘. 데이터는 랜덤한 데이터로 생성해 줘

* `read_excel, create_excel_with_formatting` 함수 활용 예

읽은 직원정보를 서식이 있는 엑셀 파일로 만들어 줘

7) 엑셀 파일 읽고 쓰기

[응용 프롬프트 1] 프로젝트 관리 엑셀 파일 자동 생성

- (문제) 현재 폴더내에 무분별하게 관리되고 있는 여러 프로젝트의 진행 현황, 전체적인 관리현황, 자산파악, 리소스 재할용 여부 파악이 어려움
- (해결방안) 이러한 문제가 가능하도록 폴더 구조와 파일 정보를 자동으로 수집하여 엑셀로 정리해 주는 도구 생성
- 프롬프트를 작성(MCP 프롬프트.md 파일 참조)

[응용 프롬프트 2] 프로젝트 완료보고서 자동 생성

- 프롬프트를 작성(MCP 프롬프트.md 파일 참조)

8) 회의록 자동 생성하기

- 회의록 템플릿 자동 생성 실습 : 프롬프트 입력
- 이전 회의 요약 자동화 : 프롬프트 입력

9) 다단계 파이프라인 구성: 시장조사에서 발표자료까지

9-1 단계별 자료 구성

- 1단계: 웹 검색 : 노화 치료제 개발 현황 정보를 웹에서 수집하여 엑셀로 저장
- 2단계: 엑셀: 비교표와 차트 추가
- 3단계: 워드: 요약 보고서 생성
- 4단계: 임원 발표용 슬라이드 생성

9-2 단계별 프롬프트 구성 (실습자료.md 참고)

강의 요약

❖ 오전

- MCP의 이론 및 관련 기술에 대한 소개
- 실습 환경 구축과 에러시 대응 방안, 간단한 실습

❖ 오후

- 외부에서 MCP 서버 가져다 쓰는 방식 실습
- MCP 코드 작성 및 직접 설정 방식 위주의 예제 실습
- 반복업무 자동화 및 활용사례



감사합니다